

```
R version 4.6.0 (2026-04-24 ucrt) -- "Because it was There"
Copyright (C) 2026 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64
```

R 是自由軟體，不提供任何擔保。  
在某些條件下歡迎您將其散佈。  
用 'license()' 或 'licence()' 來獲得散佈的詳細條件。

R 是個協作計劃，有許多人為之做出了貢獻。  
用 'contributors()' 來看詳細的情況以及  
用 'citation()' 會告訴您如何在出版品中正確地參照 R 或 R 套件。

用 'demo()' 來看一些示範程式，用 'help()' 來檢視線上輔助檔案，或  
用 'help.start()' 透過 HTML 瀏覽器來看輔助檔案。  
用 'q()' 離開 R。

[原來儲存的工作空間已還原]

```
> # CHAPTER 6 Building Empirical Models
> # Refer to § 1-2.3, 1-3, and 2-6.
> # § 6-1 Introduction to Empirical Models
> # --- 1. Setup ---
> if(!require(pacman)) install.packages("pacman")
載入需要的套件: pacman
> pacman::p_load(broom, car, ggfortify, ggplot2, goftest, HH, lmtest, mgcv,
+ nortest, olsrr, patchwork, rgl)
> # Table 6-1 Salt Concentration in Surface Streams and Roadway Area
> # --- 2. Input the actual dataset from Table 6-1 ---
> roadway_area <- c(0.19, 0.15, 0.57, 0.40, 0.70, 0.67, 0.63, 0.47,
+ 0.75, 0.60, 0.78, 0.81, 0.78, 0.69, 1.30, 1.05, 1.52, 1.06, 1.74, 1.62)
> salt_concentration <- c( 3.8, 5.9, 14.1, 10.4, 14.6, 14.5, 15.1, 11.9,
+ 15.5, 9.3, 15.6, 20.8, 14.6, 16.6, 25.6, 20.9, 29.9, 19.6, 31.3, 32.7)
> (Watershed_df <- data.frame(
+ roadway_area = roadway_area,
+ salt_concentration = salt_concentration
+ ))
   roadway_area salt_concentration
1         0.19             3.8
2         0.15             5.9
3         0.57            14.1
4         0.40            10.4
5         0.70            14.6
6         0.67            14.5
7         0.63            15.1
8         0.47            11.9
9         0.75            15.5
10        0.60             9.3
11        0.78            15.6
12        0.81            20.8
13        0.78            14.6
14        0.69            16.6
15        1.30            25.6
16        1.05            20.9
17        1.52            29.9
18        1.06            19.6
19        1.74            31.3
20        1.62            32.7
> summary(Watershed_df)
   roadway_area      salt_concentration
Min.   :0.1500   Min.   : 3.80
1st Qu.:0.5925   1st Qu.:13.55
Median :0.7250   Median :15.30
Mean    :0.8240   Mean    :17.14
3rd Qu.:1.0525   3rd Qu.:20.82
Max.    :1.7400   Max.    :32.70
> spm(Watershed_df)
> # --- 3. Fit the simple linear regression model ---
> regression_model <- lm(salt_concentration ~ roadway_area)
> # Figure 6-1 Scatter diagram of the salt concentration in surface streams
> # and roadway area data in Table 6-1.
> sp(roadway_area, salt_concentration, "xy", xlab = "Roadway area (%)",
+ ylab = "Salt conc. (mg/l)")
> # Figure 6-2 The distribution of Y for a given value of x for the salt
```

```

> # concentration-roadway area data.
> # Figure 6-3 (a) The regression plane for the model  $E(Y|x_1, x_2) = 50 +$ 
> #  $10*x_1 + 7*x_2$ . (b) The contour plot.
> f1 <- function(x1, x2) 50 + 10*x1 + 7*x2
> f1.3d <- plot3d(f1, xlab = "x1", ylab = "x2", zlab = "E(Y|x1, x2)",
+ col = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
+ "yellow", "#FF7F00", "red", "#7F0000")), xlim = c(0, 10),
+ ylim = c(0, 10))
> (lv1 <- f1(0, 0) + (0:10)*(f1(10, 10) - f1(0, 0))/10)
[1] 50 67 84 101 118 135 152 169 186 203 220
> contourLines3d(f1.3d, "z", 11, lv1)
> # Figure 6-4 (a) Three-dimensional plot of regression model  $E(Y|x_1, x_2) =$ 
> #  $50 + 10*x_1 + 7*x_2 + 5*x_1*x_2$ . (b) The contour plot.
> f2 <- function(x1, x2) 50 + 10*x1 + 7*x2 + 5*x1*x2
> f2.3d <- plot3d(f2, xlab = "x1", ylab = "x2", zlab = "E(Y|x1, x2)",
+ col = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
+ "yellow", "#FF7F00", "red", "#7F0000")), xlim = c(0, 10),
+ ylim = c(0, 10))
> (lv2 <- f2(0, 0) + (0:10)*(f2(10, 10) - f2(0, 0))/10)
[1] 50 117 184 251 318 385 452 519 586 653 720
> contourLines3d(f2.3d, "z", 11, lv2, 10000)
> # Figure 6-5 (a) Three-dimensional plot of the regression model  $E(Y|x_1, x_2)$ 
> #  $= 800 + 10*x_1 + 7*x_2 - 8.5*x_1^2 - 5*x_2^2 + 4*x_1*x_2$ . (b) The contour plot.
> f3 <- function(x1, x2) 800 + 10*x1 + 7*x2 - 8.5*x1^2 - 5*x2^2 + 4*x1*x2
> f3.3d <- plot3d(f3, xlab = "x1", ylab = "x2", zlab = "E(Y|x1, x2)",
+ col = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
+ "yellow", "#FF7F00", "red", "#7F0000")), xlim = c(0, 10),
+ ylim = c(0, 10))
> (lv3 <- f3(10, 10) + (0:10)*(f3(0, 0) - f3(10, 10))/10)
[1] 20 98 176 254 332 410 488 566 644 722 800
> contourLines3d(f3.3d, "z", 11, lv3, 10000)
> # § 6-2 Simple Linear Regression
> # Refer to § 6-1.
> # --- 1. Setup ---
> if(!require(pacman)) install.packages("pacman")
> pacman::p_load(broom, car, ggfortify, ggplot2, goftest, HH, lmtest, mgcv,
+ nortest, olsrr, patchwork, rgl)
> # --- 2. Input the actual dataset from Table 6-1 ---
> roadway_area <- c(0.19, 0.15, 0.57, 0.40, 0.70, 0.67, 0.63, 0.47,
+ 0.75, 0.60, 0.78, 0.81, 0.78, 0.69, 1.30, 1.05, 1.52, 1.06, 1.74, 1.62)
> salt_concentration <- c(3.8, 5.9, 14.1, 10.4, 14.6, 14.5, 15.1, 11.9,
+ 15.5, 9.3, 15.6, 20.8, 14.6, 16.6, 25.6, 20.9, 29.9, 19.6, 31.3, 32.7)
> (Watershed_df <- data.frame(
+ roadway_area = roadway_area,
+ salt_concentration = salt_concentration
+ ))
  roadway_area salt_concentration
1         0.19              3.8
2         0.15              5.9
3         0.57             14.1
4         0.40             10.4
5         0.70             14.6
6         0.67             14.5
7         0.63             15.1
8         0.47             11.9
9         0.75             15.5
10        0.60              9.3
11        0.78             15.6
12        0.81             20.8
13        0.78             14.6
14        0.69             16.6
15        1.30             25.6
16        1.05             20.9
17        1.52             29.9
18        1.06             19.6
19        1.74             31.3
20        1.62             32.7
> summary(Watershed_df)
  roadway_area      salt_concentration
Min.   :0.1500   Min.   : 3.80
1st Qu.:0.5925   1st Qu.:13.55
Median :0.7250   Median :15.30
Mean   :0.8240   Mean   :17.14
3rd Qu.:1.0525   3rd Qu.:20.82

```

```

Max.      :1.7400    Max.      :32.70
> spm(Watershed_df)
> # --- 3. Fit the simple linear regression model ---
> regression_model <- lm(salt_concentration ~ roadway_area)
> # Figure 6-1 Scatter diagram of the salt concentration in surface streams
> # and roadway area data in Table 6-1.
> sp(roadway_area, salt_concentration, "xy", xlab = "Roadway area (%)",
+ ylab = "Salt conc. (mg/l)")
> # Figure 6-2 The distribution of Y for a given value of x for the salt
> # concentration-roadway area data.
> # $ 6-2.1 Least Squares Estimation
> # Figure 6-6 Deviations of the data from the estimated regression model.
> regresiplot(roadway_area, salt_concentration, xlab = "Roadway area (%)",
+ ylab = "Salt Conc. (mg/l)")
> regresiplot(roadway_area, salt_concentration, T,
+ xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
> regresiplot(roadway_area, salt_concentration, "square",
+ xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
> # EXAMPLE 6-1 Salt Concentration and Roadway Data
> # Refer to Table 6-2.
> # Figure 6-7 Scatter diagram of salt concentration y versus roadway area x
> # and the fitted regression model.
> # ols_regress(salt_concentration ~ roadway_area, Watershed)
> # (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
> # summary(lm.Watershed)
> # coef(lm.Watershed)
> # (newdata <- data.frame(roadway_area = 1.25))
> # predict(lm.Watershed, newdata, T, NULL, Inf, "c")
> # predict(lm.Watershed, newdata, T, NULL, Inf, "p")
> # Table 6-2 Minitab Regression Analysis Output for Salt Concentration and
> # Roadway Data
> ols_regress(salt_concentration ~ roadway_area, Watershed)

```

#### Model Summary

|                |       |           |        |
|----------------|-------|-----------|--------|
| R              | 0.975 | RMSE      | 1.699  |
| R-Squared      | 0.951 | MSE       | 2.886  |
| Adj. R-Squared | 0.949 | Coef. Var | 10.450 |
| Pred R-Squared | 0.940 | AIC       | 83.954 |
| MAE            | 1.312 | SBC       | 86.941 |

RMSE: Root Mean Square Error  
 MSE: Mean Square Error  
 MAE: Mean Absolute Error  
 AIC: Akaike Information Criteria  
 SBC: Schwarz Bayesian Criteria

#### ANOVA

|            | Sum of Squares | DF | Mean Square | F      | Sig.   |
|------------|----------------|----|-------------|--------|--------|
| Regression | 1130.149       | 1  | 1130.149    | 352.46 | 0.0000 |
| Residual   | 57.716         | 18 | 3.206       |        |        |
| Total      | 1187.866       | 19 |             |        |        |

#### Parameter Estimates

| model        | Beta   | Std. Error | Std. Beta | t      | Sig.  | lower  | upper  |
|--------------|--------|------------|-----------|--------|-------|--------|--------|
| (Intercept)  | 2.677  | 0.868      |           | 3.084  | 0.006 | 0.853  | 4.500  |
| roadway_area | 17.547 | 0.935      | 0.975     | 18.774 | 0.000 | 15.583 | 19.510 |

```
> (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
```

```

Call:
lm(formula = salt_concentration ~ roadway_area, data = Watershed)

```

```

Coefficients:
(Intercept)  roadway_area
      2.677      17.547

```

```
> summary(lm.Watershed)
```

```
Call:
lm(formula = salt_concentration ~ roadway_area, data = Watershed)
```

```
Residuals:
```

```
      Min       1Q   Median       3Q      Max
-3.9045 -0.9912  0.0900  1.0747  3.9107
```

```
Coefficients:
```

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    2.6765      0.8680   3.084  0.00641 **
roadway_area   17.5467      0.9346  18.774 2.86e-13 ***
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.791 on 18 degrees of freedom
Multiple R-squared:  0.9514,    Adjusted R-squared:  0.9487
F-statistic: 352.5 on 1 and 18 DF,  p-value: 2.863e-13
```

```
> coef(lm.Watershed)
(Intercept) roadway_area
 2.676546    17.546667
> anova(lm.Watershed)
Analysis of Variance Table
```

```
Response: salt_concentration
      Df Sum Sq Mean Sq F value    Pr(>F)
roadway_area  1 1130.15  1130.15   352.46 2.863e-13 ***
Residuals    18   57.72    3.21
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.Watershed, "roadway_area")
```

```
Linear hypothesis test:
roadway_area = 0
```

```
Model 1: restricted model
Model 2: salt_concentration ~ roadway_area
```

```
      Res.Df    RSS Df Sum of Sq      F      Pr(>F)
1         19 1187.87
2         18   57.72  1    1130.2 352.46 2.863e-13 ***
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
> fitted(lm.Watershed)
      1      2      3      4      5      6      7      8
 6.010413  5.308546 12.678147  9.695213 14.959213 14.432813 13.730947 10.923480
      9     10     11     12     13     14     15     16
15.836547 13.204547 16.362947 16.889347 16.362947 14.783747 25.487214 21.100547
     17     18     19     20
29.347480 21.276013 33.207747 31.102147
> predict(lm.Watershed, Watershed, T)
```

```
$fit
      1      2      3      4      5      6      7      8
 6.010413  5.308546 12.678147  9.695213 14.959213 14.432813 13.730947 10.923480
      9     10     11     12     13     14     15     16
15.836547 13.204547 16.362947 16.889347 16.362947 14.783747 25.487214 21.100547
     17     18     19     20
29.347480 21.276013 33.207747 31.102147
```

```
$se.fit
      1      2      3      4      5      6      7      8
0.7151536 0.7464234 0.4654887 0.5633499 0.4168385 0.4254875 0.4395443 0.5194137
      9     10     11     12     13     14     15     16
0.4063329 0.4518333 0.4025097 0.4006172 0.4025097 0.4195332 0.5985353 0.4527024
     17     18     19     20
0.7638561 0.4571381 0.9451273 0.8448711
```

```
$df
[1] 18
```

```
$residual.scale
[1] 1.790659
```

```
> resid(lm.Watershed)
```

```

1          2          3          4          5          6
-2.21041306 0.59145363 1.42185344 0.70478685 -0.35921328 0.06718673
7          8          9          10         11         12
1.36905342 0.97652015 -0.33654663 -3.90454657 -0.76294665 3.91065334
13         14         15         16         17         18
-1.76294665 1.81625339 0.11278646 -0.20054677 0.55251970 -1.67601344
19         20
-1.90774707 1.59785299
> confint(lm.Watershed)
              2.5 %      97.5 %
(Intercept)  0.852938  4.500155
roadway_area 15.583083 19.510251
> (newdata <- data.frame(roadway_area = 1.25))
   roadway_area
1           1.25
> predict(lm.Watershed, newdata, T, NULL, Inf, "c")
$fit
      fit      lwr      upr
1 24.60988 23.42356 25.7962

$se.fit
[1] 0.5646664

$df
[1] 18

$residual.scale
[1] 1.790659

> predict(lm.Watershed, newdata, T, NULL, Inf, "p")
$fit
      fit      lwr      upr
1 24.60988 20.66523 28.55453

$se.fit
[1] 0.5646664

$df
[1] 18

$residual.scale
[1] 1.790659

> ci.plot(lm.Watershed, main = "95% confidence and prediction intervals",
+ xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
> # EXAMPLE 6-1 (continued)
> # Refer to Table 6-2.
> # ols_regress(salt_concentration ~ roadway_area, Watershed)
> # (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
> # summary(lm.Watershed)
> # anova(lm.Watershed)
> # lht(lm.Watershed, "roadway_area")
> # EXAMPLE 6-1 (continued)
> # Refer to Table 6-2.
> # ols_regress(salt_concentration ~ roadway_area, Watershed)
> # (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
> # summary(lm.Watershed)
> # --- A. R-Squared and Adjusted R-Squared ---
> # 1. Fit the ordinary least-squares simple linear regression model
> regression_model <- lm(salt_concentration ~ roadway_area)
> model_summary <- summary(regression_model)
> # 2. Extract the R-squared and adjusted R-squared parameters
> r_squared <- model_summary$r.squared
> adj_r_squared <- model_summary$adj.r.squared
> # 3. Print out the structured variance explanation report
> cat("=====\n")
=====
> cat(" MODEL EXPLANATORY POWER VARIANCE SUMMARY \n")
MODEL EXPLANATORY POWER VARIANCE SUMMARY
> cat("=====\n")
=====
> cat(sprintf("Sample Size (n): %d\n",
+ length(salt_concentration)))
Sample Size (n): 20

```

```

> cat(sprintf("Coefficient of Determination (R^2): %s\n",
+ signif(r_squared, 4)))
Coefficient of Determination (R^2): 0.9514
> cat(sprintf("Adjusted R-Squared (Adj R^2): %s\n",
+ signif(adj_r_squared, 4)))
Adjusted R-Squared (Adj R^2): 0.9487
> cat("=====\n")
=====
> # § 6-2.2 Testing Hypotheses in Simple Linear Regression
> # Refer to § 6-2.1.
> # Figure 6-8 The hypothesis  $H_0: \beta_1 = 0$  is not rejected.
> # Figure 6-9 The hypothesis  $H_0: \beta_1 = 0$  is rejected.
> # EXAMPLE 6-2 Salt Concentration and Roadway Data
> # Refer to Table 6-2.
> # ols_regress(salt_concentration ~ roadway_area, Watershed)
> # (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
> # summary(lm.Watershed)
> # Table 6-3 Analysis of Variance for Testing Significance of Regression
> # EXAMPLE 6-2 (continued)
> # Refer to Table 6-2.
> # ols_regress(salt_concentration ~ roadway_area, Watershed)
> # (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
> # summary(lm.Watershed)
> # § 6-2.3 Confidence Intervals in Simple Linear Regression
> # Refer to § 6-2.1.
> # EXAMPLE 6-3 Salt Concentration and Roadway Data
> # Refer to Table 6-2.
> # ols_regress(salt_concentration ~ roadway_area, Watershed)
> # confint(lm.Watershed)
> # Confidence Interval on the Mean Response
> # EXAMPLE 6-4 Salt Concentration and Roadway Data
> # Refer to Table 6-2.
> # ci.plot(lm.Watershed, main = "95% confidence and prediction intervals",
> # xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
> # (newdata <- data.frame(roadway_area = 1.25))
> # predict(lm.Watershed, newdata, T, NULL, Inf, "c")
> # Figure 6-10 Scatter diagram of salt concentration and roadway area from
> # Example 6-1 with fitted regression line and 95% confidence limits on
> #  $\mu_{Y|x_0}$ .
> # § 6-2.4 Prediction of a Future Observation
> # Refer to § 6-2.1 and 6-2.3.
> # EXAMPLE 6-5 Salt Concentration and Roadway Data
> # Refer to Table 6-2.
> # (newdata <- data.frame(roadway_area = 1.25))
> # predict(lm.Watershed, newdata, T, NULL, Inf, "p")
> # ci.plot(lm.Watershed, main = "95% confidence and prediction intervals",
> # xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
> # Figure 6-11 Scatter diagram of salt concentration-roadway area data from
> # Example 6-1 with fitted regression line, 95% prediction limits (outer
> # lines), and 95% confidence limits on  $\mu_{Y|x_0}$ .
> # § 6-2.5 Checking Model Adequacy
> # Refer to § 6-2.1.
> # --- A. Diagnostic Plots ---
> # 1. Configure the plotting region to a 2-row, 2-column matrix grid layout.
> par(mfrow = c(2, 2))
> # 2. Generate the 4 classic diagnostic plots automatically.
> # Plot 1: Residuals vs. Fitted (Checks linearity and constant conditional
> # variance.)
> # Plot 2: Normal Q-Q (Checks conditional normality assumption for errors)
> # Plot 3: Scale-Location (Checks homoscedasticity via the absolute value of
> # standardized residuals)
> # Plot 4: Cook's Distance (Identifies influential points via Cook's D
> # boundaries)
> plot(regression_model,
+ which = 1:4,
+ pch = 19,
+ col = "black",
+ sub.caption = "")
> # 3. Reset the graphics grid parameter back to standard default.
> par(mfrow = c(1, 1))
> # --- B. Diagnostic Plots (ggplot2) ---
> # 1. Use broom::augment() to convert internal model diagnostics into a
> # clean data frame.
> diagnostic_data <- augment(regression_model)

```

```

> diagnostic_data$obs_index <- 1:nrow(diagnostic_data) # Add individual
> # observation index numbers.
> # 2. Handcraft the 4 classic diagnostic charts using standard modern
> # syntax. This isolates line thickness and completely bypasses aesthetic
> # warning blocks.
> # Plot 1: Residuals vs. Fitted
> p1 <- ggplot(diagnostic_data, aes(x = .fitted, y = .resid)) +
+ geom_point(color = "black", size = 2) +
+ geom_smooth(method = "loess", formula = y ~ x, color = "red", se = FALSE,
+ linewidth = 0.8) +
+ geom_hline(yintercept = 0, linetype = "dashed", color = "black",
+ linewidth = 0.5) +
+ labs(title = "Residuals vs. Fitted", x = "Fitted values",
+ y = "Residuals")
> # Plot 2: Normal Q-Q
> p2 <- ggplot(diagnostic_data, aes(sample = .std.resid)) +
+ stat_qq(color = "black", size = 2) +
+ stat_qq_line(linetype = "dashed", color = "red", linewidth = 0.8) +
+ labs(title = "Normal Q-Q", x = "Theoretical Quantiles",
+ y = "Standardized Residuals")
> # Plot 3: Scale-Location
> p3 <- ggplot(diagnostic_data,
+ aes(x = .fitted, y = sqrt(abs(.std.resid)))) +
+ geom_point(color = "black", size = 2) +
+ geom_smooth(method = "loess", formula = y ~ x, color = "red", se = FALSE,
+ linewidth = 0.8) +
+ labs(title = "Scale-Location", x = "Fitted values",
+ y = "sqrt(|Standardized Residuals|)")
> # Plot 4: Cook's Distance (Bar plot format for tracking influential
> # observations)
> p4 <- ggplot(diagnostic_data, aes(x = obs_index, y = .cooks_d)) +
+ geom_col(fill = "black", width = 0.5) +
+ labs(title = "Cook's Distance", x = "Obs. Number", y = "Cook's distance")
> # 3. Assemble the separate quadrants into a single 2 x 2 matrix grid
> # utilizing patchwork math.
> modern_diagnostic_grid <- (p1 | p2) / (p3 | p4)
> # 4. Render the final composite graphics output window safely without
> # warnings
> print(modern_diagnostic_grid)
> # Figure 6-12 Patterns for residual plots: (a) satisfactory, (b) funnel,
> # (c) double bow, (d) nonlinear. Horizontal axis may be time, yihat, or xi.
> # EXAMPLE 6-6 Salt Concentration and Roadway Data
> # Refer to Table 6-2.
> # Figure 6-13 Normal probability plot of residuals from the salt
> # concentration-roadway area regression model.
> # Figure 6-14 Plot of residuals versus fitted values for the salt
> # concentration-roadway area regression model.
> # 1. Extract diagnostics: fitted values and ordinary residuals
> fitted_values <- predict(regression_model)
> residuals_e <- resid(regression_model)
> observation_order <- 1:length(residuals_e)
> # 2. Set up a 1-row, 2-column plotting layout
> par(mfrow = c(1, 2))
> # Plot A: Residuals vs. Fitted Values (Checks Constant Conditional Variance
> # & Linearity)
> plot(fitted_values, residuals_e,
+ main = "Residuals vs. Fitted Values",
+ xlab = "Fitted Values (y)",
+ ylab = "Residuals (e)",
+ pch = 19, col = "black")
> abline(h = 0, lty = 2, col = "black") # Add baseline reference
> # Plot B: Residuals vs. Observation Order (Checks Independence /
> # Autocorrelation)
> plot(observation_order, residuals_e,
+ main = "Residuals vs. Time/Order",
+ xlab = "Observation Order",
+ ylab = "Residuals (e)",
+ type = "b", pch = 19, col = "black") # "b" connects points with lines
> abline(h = 0, lty = 2, col = "black") # Add baseline reference
> # Reset plotting layout parameters back to default
> par(mfrow = c(1, 1))
> # 3. Extract specific diagnostic metrics for summary output
> summary_stats <- summary(regression_model)
> ms_e <- summary_stats$sigma^2 # Mean Square Error (estimated variance  $\sigma^2$ )

```

```

> cat("=====\n")
=====
> cat(" RESIDUAL VARIANCE DIAGNOSTIC SUMMARY \n")
RESIDUAL VARIANCE DIAGNOSTIC SUMMARY
> cat("=====\n")
=====
> cat(sprintf("Sample Size (n): %d\n",
+ length(salt_concentration)))
Sample Size (n): 20
> cat(sprintf("Mean Square Error (MS_E or sigma^2): %s\n", signif(ms_e, 4)))
Mean Square Error (MS_E or sigma^2): 3.206
> cat(sprintf("Maximum Absolute Residual value: %s\n",
+ signif(max(abs(residuals_e)), 4)))
Maximum Absolute Residual value: 3.911
> cat("=====\n")
=====
> # 4. Generate the Normal Probability Plot
> qqnorm(residuals_e,
+ main = "Normal Probability Plot of Residuals",
+ xlab = "Theoretical Quantiles",
+ ylab = "Sample Quantiles (Residuals)",
+ pch = 19, col = "black")
> # Add a reference line passing through the first and third quartiles
> qqline(residuals_e, lty = 2, col = "black")
> # 5. Extract and print descriptive metrics to verify shape properties
> # Calculate sample skewness and kurtosis manually for the summary report
> n <- length(residuals_e)
> mean_e <- mean(residuals_e)
> sd_e <- sd(residuals_e)
> skewness <- (sum((residuals_e - mean_e)^3) / n) /
+ (sum((residuals_e - mean_e)^2) / n)^(3/2)
> kurtosis <- (sum((residuals_e - mean_e)^4) / n) /
+ (sum((residuals_e - mean_e)^2) / n)^2
> cat("=====\n")
=====
> cat(" RESIDUAL NORMALITY SHAPE SUMMARY \n")
RESIDUAL NORMALITY SHAPE SUMMARY
> cat("=====\n")
=====
> cat(sprintf("Sample Size (n): %d\n", n))
Sample Size (n): 20
> cat(sprintf("Residual Mean (should be zero): %s\n", signif(mean_e, 4)))
Residual Mean (should be zero): 8.188e-17
> cat(sprintf("Residual Sample Skewness: %s\n", signif(skewness, 4)))
Residual Sample Skewness: -0.1124
> cat(sprintf("Residual Sample Kurtosis: %s\n", signif(kurtosis, 4)))
Residual Sample Kurtosis: 3.288
> cat("=====\n")
=====
> # 6. Perform the formal Shapiro-Wilk normality test
> shapiro_results <- shapiro.test(residuals_e)
> # 7. Print out the structured normality report
> cat("=====\n")
=====
> cat(" SHAPIRO-WILK NORMALITY TEST REPORT \n")
SHAPIRO-WILK NORMALITY TEST REPORT
> cat("=====\n")
=====
> cat(sprintf("Sample Size (n): %d\n", length(residuals_e)))
Sample Size (n): 20
> cat(sprintf("Test Statistic (W): %s\n",
+ signif(shapiro_results$statistic, 4)))
Test Statistic (W): 0.9765
> cat(sprintf("Exact Test P-Value: %s\n",
+ signif(shapiro_results$p.value, 4)))
Exact Test P-Value: 0.8808
> cat("=====\n")
=====
> # 8. Extract ordinary residuals and compute standardized residuals
> # rstandard() accounts for the leverage profile h_ii of each individual
> # observation: d_i = e_i / sqrt(MS_E * (1 - h_ii))
> standardized_res <- rstandard(regression_model)
> # 9. Create a clean matrix layout for the output report
> diagnostic_table <- data.frame(

```



```

+ Observation = 1:length(standardized_res),
+ Roadway_Area = roadway_area,
+ Salt_Conc = salt_concentration,
+ Standardized_Residual = standardized_res
+ )
> # 10. Print the formatted table matching 4 significant figures
> cat("=====\n")
=====
> cat(" STANDARDIZED RESIDUAL DIAGNOSTIC TABLE \n")
STANDARDIZED RESIDUAL DIAGNOSTIC TABLE
> cat("=====\n")
=====
> for(i in 1:nrow(diagnostic_table)) {
+ cat(sprintf("Obs %2d | X: %5.2f | Y: %5.1f | Standardized Residual:
+ %s\n",
+ diagnostic_table$Observation[i],
+ diagnostic_table$Roadway_Area[i],
+ diagnostic_table$Salt_Conc[i],
+ signif(diagnostic_table$Standardized_Residual[i], 4)))
+ }
Obs 1 | X: 0.19 | Y: 3.8 | Standardized Residual:
-1.346
Obs 2 | X: 0.15 | Y: 5.9 | Standardized Residual:
0.3634
Obs 3 | X: 0.57 | Y: 14.1 | Standardized Residual:
0.8223
Obs 4 | X: 0.40 | Y: 10.4 | Standardized Residual:
0.4146
Obs 5 | X: 0.70 | Y: 14.6 | Standardized Residual:
-0.2063
Obs 6 | X: 0.67 | Y: 14.5 | Standardized Residual:
0.03863
Obs 7 | X: 0.63 | Y: 15.1 | Standardized Residual:
0.7887
Obs 8 | X: 0.47 | Y: 11.9 | Standardized Residual:
0.5698
Obs 9 | X: 0.75 | Y: 15.5 | Standardized Residual:
-0.193
Obs 10 | X: 0.60 | Y: 9.3 | Standardized Residual:
-2.253
Obs 11 | X: 0.78 | Y: 15.6 | Standardized Residual:
-0.4373
Obs 12 | X: 0.81 | Y: 20.8 | Standardized Residual:
2.241
Obs 13 | X: 0.78 | Y: 14.6 | Standardized Residual:
-1.01
Obs 14 | X: 0.69 | Y: 16.6 | Standardized Residual:
1.043
Obs 15 | X: 1.30 | Y: 25.6 | Standardized Residual:
0.06683
Obs 16 | X: 1.05 | Y: 20.9 | Standardized Residual:
-0.1158
Obs 17 | X: 1.52 | Y: 29.9 | Standardized Residual:
0.3412
Obs 18 | X: 1.06 | Y: 19.6 | Standardized Residual:
-0.9681
Obs 19 | X: 1.74 | Y: 31.3 | Standardized Residual:
-1.254
Obs 20 | X: 1.62 | Y: 32.7 | Standardized Residual:
1.012
> cat("=====\n")
=====
> # 11. Compute the square root of the absolute standardized residuals
> sqrt_abs_std_res <- sqrt(abs(standardized_res))
> # 12. Generate the Scale-Location plot
> plot(fitted_values, sqrt_abs_std_res,
+ main = "Scale-Location Plot",
+ xlab = "Fitted Values (y)",
+ ylab = "Square Root of |Standardized Residuals|",
+ pch = 19, col = "black")
> # Add a smooth trend line (lowess) to reveal structural variance patterns
> lines(lowess(fitted_values, sqrt_abs_std_res), lty = 1, lwd = 2,
+ col = "black")
> # 13. Calculate Cook's Distance for each individual observation

```

```

> cooks_d <- cooks.distance(regression_model)
> # 14. Establish standard threshold rules for influential points
> # A common screening threshold rule is 4 / n, where n is the sample size
> sample_size <- length(salt_concentration)
> threshold <- 4 / sample_size
> # 15. Print the formatted diagnostics table matching 4 significant figures
> cat("=====\n")
=====
> cat(" COOK'S DISTANCE DIAGNOSTIC TABLE \n")
COOK'S DISTANCE DIAGNOSTIC TABLE
> cat("=====\n")
=====
> cat(sprintf("Sample Size (n): %d | Influential Threshold (4/n): %s\n",
+ sample_size, signif(threshold, 4)))
Sample Size (n): 20 | Influential Threshold (4/n): 0.2
> cat("-----\n")
-----
> for(i in 1:sample_size) {
+ status <- if(cooks_d[i] > threshold) " * INFLUENTIAL *" else "Normal"
+ cat(sprintf("Obs %2d | Cook's D: %s \t| Status: %s\n",
+ i,
+ signif(cooks_d[i], 4),
+ status))
+ }
Obs  1 | Cook's D: 0.172          | Status: Normal
Obs  2 | Cook's D: 0.01388          | Status: Normal
Obs  3 | Cook's D: 0.0245           | Status: Normal
Obs  4 | Cook's D: 0.009443         | Status: Normal
Obs  5 | Cook's D: 0.001219         | Status: Normal
Obs  6 | Cook's D: 4.464e-05         | Status: Normal
Obs  7 | Cook's D: 0.01994          | Status: Normal
Obs  8 | Cook's D: 0.01492          | Status: Normal
Obs  9 | Cook's D: 0.001011         | Status: Normal
Obs 10 | Cook's D: 0.1726           | Status: Normal
Obs 11 | Cook's D: 0.005087         | Status: Normal
Obs 12 | Cook's D: 0.1323           | Status: Normal
Obs 13 | Cook's D: 0.02716          | Status: Normal
Obs 14 | Cook's D: 0.03161          | Status: Normal
Obs 15 | Cook's D: 0.0002809        | Status: Normal
Obs 16 | Cook's D: 0.0004575        | Status: Normal
Obs 17 | Cook's D: 0.01294          | Status: Normal
Obs 18 | Cook's D: 0.03267          | Status: Normal
Obs 19 | Cook's D: 0.3038           | Status:  * INFLUENTIAL *
Obs 20 | Cook's D: 0.1467          | Status: Normal
> cat("=====\n")
=====
> # Figure 6-15 Effect of an outlier.
> # --- C. Formal Partial F-Test ---
> # 1. Fit the Reduced Model (First-Order Linear)
> model_reduced <- lm(salt_concentration ~ roadway_area)
> # 2. Fit the Full Model (Second-Order Polynomial / Quadratic)
> model_full <- lm(salt_concentration ~ poly(roadway_area, 2, raw = TRUE))
> # 3. Perform the formal Partial F-Test
> partial_f_test <- anova(model_reduced, model_full)
> # 4. Extract specific diagnostic metrics for the 4-significant-figures
> # report
> # The anova() output contains rows for both models. Row 2 captures the
> # partial differences.
> df_reduced <- partial_f_test$Res.Df[1]
> df_full <- partial_f_test$Res.Df[2]
> df_diff <- partial_f_test$Df[2] # Degrees of freedom for the extra term
> # (2 - 1 = 1)
> ss_reduced <- partial_f_test$RSS[1]
> ss_full <- partial_f_test$RSS[2]
> ss_diff <- partial_f_test$`Sum of Sq`[2] # Extra Sum of Squares:
> # SS_Reduced - SS_Full
> f_statistic <- partial_f_test$F[2]
> p_value <- partial_f_test$`Pr(>F)`[2]
> # 5. Derive the exact F critical value for comparison
> alpha <- 0.05
> f_critical <- qf(1 - alpha, df_diff, df_full)
> # 6. Print out the structured Partial F-Test Report
> cat("=====\n")
=====

```

```

> cat(" FORMAL PARTIAL F-TEST REPORT \n")
FORMAL PARTIAL F-TEST REPORT
> cat("=====\n")
=====
> cat(sprintf("Reduced Model Residual Df: %d\n", df_reduced))
Reduced Model Residual Df: 18
> cat(sprintf("Full Model Residual Df: %d\n", df_full))
Full Model Residual Df: 17
> cat(sprintf("Reduced Model Residual SS (SS_E): %s\n",
+ signif(ss_reduced, 4)))
Reduced Model Residual SS (SS_E): 57.72
> cat(sprintf("Full Model Residual SS (SS_E): %s\n",
+ signif(ss_full, 4)))
Full Model Residual SS (SS_E): 56.98
> cat(sprintf("Extra Sum of Squares (SS_Drop): %s\n",
+ signif(ss_diff, 4)))
Extra Sum of Squares (SS_Drop): 0.7373
> cat("\n--- Test Statistic and Rejection Threshold ---\n")

--- Test Statistic and Rejection Threshold ---
> cat(sprintf("Calculated Partial F-Statistic (f0): %s\n",
+ signif(f_statistic, 4)))
Calculated Partial F-Statistic (f0): 0.22
> cat(sprintf("F Rejection Critical Value (f_alpha): %s\n",
+ signif(f_critical, 4)))
F Rejection Critical Value (f_alpha): 4.451
> cat(sprintf("Exact Test P-Value: %.4f\n", p_value))
Exact Test P-Value: 0.6450
> cat("=====\n")
=====
> # --- D. Quadratic T-Test and Partial F-Test ---
> # 1. Fit the Full Model (Second-Order Polynomial / Quadratic)
> model_full <- lm(salt_concentration ~ poly(roadway_area, 2, raw = TRUE))
> summary_full <- summary(model_full)
> # 2. Extract the t-statistic for the quadratic term (beta2_hat)
> # In poly(roadway_area, 2, raw = TRUE), the terms are named 'X' and 'X^2'
> quadratic_t0 <-
+ summary_full$coefficients["poly(roadway_area, 2, raw = TRUE)2", "t value"]
> # 3. Square the extracted t-statistic
> squared_t0 <- quadratic_t0^2
> # 4. Re-fit the Reduced Model and extract the Partial F-statistic (f0)
> model_reduced <- lm(salt_concentration ~ roadway_area)
> partial_f_test <- anova(model_reduced, model_full)
> partial_f0 <- partial_f_test$F[2] # Row 2 captures the partial
> # difference statistic
> # 6. Print out the structured verification report
> cat("=====\n")
=====
> cat(" QUADRATIC T-TEST AND PARTIAL F-TEST \n")
QUADRATIC T-TEST AND PARTIAL F-TEST
> cat("=====\n")
=====
> cat(sprintf("Quadratic Coefficient t-Test Statistic (t0): %s\n",
+ signif(quadratic_t0, 4)))
Quadratic Coefficient t-Test Statistic (t0): -0.469
> cat(sprintf("Square of the Quadratic t-Statistic (t0^2): %s\n",
+ signif(squared_t0, 4)))
Square of the Quadratic t-Statistic (t0^2): 0.22
> cat(sprintf("Calculated Partial F-Test Statistic (f0): %s\n",
+ signif(partial_f0, 4)))
Calculated Partial F-Test Statistic (f0): 0.22
> cat("\n--- Identity Check ---\n")

--- Identity Check ---
> cat(sprintf("Is t0^2 exactly equal to f0? %s\n",
+ isTRUE(all.equal(squared_t0, partial_f0))))
Is t0^2 exactly equal to f0? TRUE
> cat("=====\n")
=====
> # --- E. Second-Order Polynomial Regression Model ---
> # 1. Create a unified data frame for plotting
> plot_data_poly <- data.frame(X = roadway_area, Y = salt_concentration)
> # 2. Fit the second-order polynomial (quadratic) regression model
> # poly(X, 2, raw = TRUE) sets up the terms: X + X^2

```

```

> quadratic_model <- lm(Y ~ poly(X, 2, raw = TRUE), data = plot_data_poly)
> quad_summary <- summary(quadratic_model)
> # 3. Generate a dense grid for smooth parabolic curve rendering
> dense_x <- seq(min(plot_data_poly$X), max(plot_data_poly$X),
+ length.out = 200)
> prediction_grid <- data.frame(X = dense_x)
> # Extract 95% Confidence and Prediction Intervals for the quadratic fit
> ci_quad <- predict(quadratic_model, newdata = prediction_grid,
+ interval = "confidence", level = 0.95)
> pi_quad <- predict(quadratic_model, newdata = prediction_grid,
+ interval = "prediction", level = 0.95)
> # Combine boundaries into a single tracking data frame
> interval_bands_quad <- data.frame(
+ X = dense_x,
+ Fitted = ci_quad[, "fit"],
+ CI_Lwr = ci_quad[, "lwr"],
+ CI_Upr = ci_quad[, "upr"],
+ PI_Lwr = pi_quad[, "lwr"],
+ PI_Upr = pi_quad[, "upr"]
+ )
> # 4. Calculate exact target metrics at x0 = 1.25% for comparison
> target_x0 <- data.frame(X = 1.25)
> target_pi <- predict(quadratic_model, newdata = target_x0,
+ interval = "prediction", level = 0.95)
> quad_lwr <- target_pi[1, "lwr"]
> quad_upr <- target_pi[1, "upr"]
> # 5. Build the layered polynomial ggplot visualization
> polynomial_plot <- ggplot() +
+ # Layer A: Plot the raw observation scatter points
+ geom_point(data = plot_data_poly, aes(x = X, y = Y), color = "black",
+ size = 2.5) +
+ # Layer B: Add 95% Prediction Intervals (Outer dashed lines)
+ geom_line(data = interval_bands_quad, aes(x = X, y = PI_Lwr),
+ linetype = "dashed", color = "black", linewidth = 0.8) +
+ geom_line(data = interval_bands_quad, aes(x = X, y = PI_Upr),
+ linetype = "dashed", color = "black", linewidth = 0.8) +
+ # Layer C: Add 95% Confidence Intervals (Inner shaded ribbon)
+ geom_ribbon(data = interval_bands_quad,
+ aes(x = X, ymin = CI_Lwr, ymax = CI_Upr), alpha = 0.15,
+ fill = "black") +
+ # Layer D: Superimpose the smooth quadratic fitted model curve
+ geom_line(data = interval_bands_quad, aes(x = X, y = Fitted),
+ linetype = "solid", color = "black", linewidth = 1.0) +
+ # Layer E: Example 6-5 comparative reference markers at x0 = 1.25%
+ geom_vline(xintercept = 1.25, linetype = "dotted", color = "black",
+ linewidth = 0.8) +
+ annotate("point", x = 1.25, y = quad_lwr, color = "black", size = 3,
+ shape = 21, fill = "white", stroke = 1.5) +
+ annotate("point", x = 1.25, y = quad_upr, color = "black", size = 3,
+ shape = 21, fill = "white", stroke = 1.5) +
+ annotate("text", x = 1.29, y = quad_lwr,
+ label = paste("Quad Lwr =", signif(quad_lwr, 4)), hjust = 0,
+ size = 3.5, fontface = "bold") +
+ annotate("text", x = 1.29, y = quad_upr,
+ label = paste("Quad Upr =", signif(quad_upr, 4)), hjust = 0,
+ size = 3.5, fontface = "bold") +
+ # Layer F: Configure labels and formatting
+ labs(
+ title = "Second-Order Polynomial Regression Model Fit",
+ subtitle = "Model: Y = 1.047 + 22.84X - 3.012X² | Shaded: 95% CI |
+ Dashed: 95% PI",
+ x = "Roadway Area (%)",
+ y = "Salt Concentration (mg/l)"
+ ) +
+ theme(
+ plot.title = element_text(face = "bold", size = 12, hjust = 0.5),
+ plot.subtitle = element_text(size = 10, hjust = 0.5, face = "italic"),
+ axis.title = element_text(face = "bold", size = 11)
+ )
> # 6. Render the final composite graphics output window
> print(polynomial_plot)
> # 7. Extract and print quadratic coefficients details for tracking
> # verification
> beta0 <- quad_summary$coefficients[1, 1]

```

```

> beta1 <- quad_summary$coefficients[2, 1]
> beta2 <- quad_summary$coefficients[3, 1]
> r2_q <- quad_summary$r.squared
> cat("--- Quadratic Model Parameter Details ---\n")
--- Quadratic Model Parameter Details ---
> cat(sprintf("Intercept (beta0_hat): %s\n", signif(beta0, 4)))
Intercept (beta0_hat): 2.009
> cat(sprintf("Linear Coeff (beta1_hat): %s\n", signif(beta1, 4)))
Linear Coeff (beta1_hat): 19.32
> cat(sprintf("Quad Coeff (beta2_hat): %s\n", signif(beta2, 4)))
Quad Coeff (beta2_hat): -0.9223
> cat(sprintf("Model Fit Quality (R^2): %s\n", signif(r2_q, 4)))
Model Fit Quality (R^2): 0.952
> # --- F. Diagnostic Plots (Second-Order Polynomial Regression Model) ---
> # 1. Fit the Full Model (Second-Order Polynomial / Quadratic)
> # poly(..., raw = TRUE) keeps the variables on the original raw scale
> model_quadratic <- lm(salt_concentration ~ poly(roadway_area, 2,
+ raw = TRUE))
> # 3. Configure the plotting region to a 1-row, 2-column matrix grid layout
> # This isolates the two primary plots needed to evaluate distribution
> # improvements
> par(mfrow = c(1, 2))
> # Plot A: Residuals vs. Fitted Plot (Checks Linearity and Homoscedasticity)
> plot(model_quadratic,
+ which = 1,
+ pch = 19,
+ col = "black",
+ sub.caption = "")
> # Plot B: Normal Q-Q Plot (Checks Conditional Error Normality)
> plot(model_quadratic,
+ which = 2,
+ pch = 19,
+ col = "black",
+ sub.caption = "")
> # Reset plotting layout parameters back to standard default
> par(mfrow = c(1, 1))
> # --- G. Lack-of-Fit Test ---
> # 1. Fit the Proposed Reduced Model (First-Order Linear Model)
> model_linear <- lm(salt_concentration ~ roadway_area)
> # 2. Fit the Saturated Model (Groups unique X values as separate factor
> # levels)
> # This model captures all available replicate information to compute Pure
> # Error
> model_saturated <- lm(salt_concentration ~ factor(roadway_area))
> # 3. Perform the formal Lack-of-Fit ANOVA Decompositions
> lof_table <- anova(model_linear, model_saturated)
> # 4. Extract components for our clean 4-significant-figures layout matrix
> # Pure Error metrics are derived from the saturated model row (Row 2)
> df_pure_error <- lof_table$Res.Df[2]
> ss_pure_error <- lof_table$RSS[2]
> ms_pure_error <- ss_pure_error / df_pure_error
> # Lack-of-Fit metrics represent the drop between the two specifications
> df_lack_of_fit <- lof_table$Df[2]
> ss_lack_of_fit <- lof_table$`Sum of Sq`[2]
> ms_lack_of_fit <- ss_lack_of_fit / df_lack_of_fit
> # Calculate the formal Lack-of-Fit F-test statistic
> f0_lof <- ms_lack_of_fit / ms_pure_error
> p_val_lof <- pf(f0_lof, df_lack_of_fit, df_pure_error, lower.tail = FALSE)
> # 5. Extract the alternative Model Comparison metric (Partial F-Test for
> # the Quadratic model)
> model_quadratic <- lm(salt_concentration ~ poly(roadway_area, 2,
+ raw = TRUE))
> quad_table <- anova(model_linear, model_quadratic)
> p_val_quad <- quad_table$`Pr(>F)`[2]
> # 6. Print the official Lack-of-Fit Report
> cat("=====\n")
=====
> cat(" FORMAL PURE ERROR LACK-OF-FIT REPORT \n")
FORMAL PURE ERROR LACK-OF-FIT REPORT
> cat("=====\n")
=====
> cat(sprintf("Lack-of-Fit Sum of Squares (SS_LF): %s\n",
+ signif(ss_lack_of_fit, 4)))
Lack-of-Fit Sum of Squares (SS_LF): 57.22

```

```

> cat(sprintf("Pure Error Sum of Squares (SS_PE): %s\n",
+ signif(ss_pure_error, 4)))
Pure Error Sum of Squares (SS_PE): 0.5
> cat(sprintf("Lack-of-Fit Degrees of Freedom (df_LF): %d\n",
+ df_lack_of_fit))
Lack-of-Fit Degrees of Freedom (df_LF): 17
> cat(sprintf("Pure Error Degrees of Freedom (df_PE): %d\n",
+ df_pure_error))
Pure Error Degrees of Freedom (df_PE): 1
> cat("\n--- Test Statistic and Rejection Threshold ---\n")

--- Test Statistic and Rejection Threshold ---
> cat(sprintf("Calculated Lack-of-Fit F-Statistic: %s\n",
+ signif(f0_lof, 4)))
Calculated Lack-of-Fit F-Statistic: 6.731
> cat(sprintf("Lack-of-Fit Test P-Value: %.4f\n", p_val_lof))
Lack-of-Fit Test P-Value: 0.2953
> cat("-----\n")
-----
> cat(sprintf("Alternative Quadratic Term P-Value: %.4f\n", p_val_quad))
Alternative Quadratic Term P-Value: 0.6450
> cat("=====\n")
=====
> # --- H. Log-Transformed Regression Model ---
> # 1. Create a data frame and compute the natural log of the response
> # variable
> plot_data_log <- data.frame(
+ X = roadway_area,
+ Y = salt_concentration,
+ LogY = log(salt_concentration) # ln(Y) transformation
+ )
> # 2. Fit the log-transformed simple linear regression model
> log_model <- lm(LogY ~ X, data = plot_data_log)
> log_summary <- summary(log_model)
> # 3. Generate a dense grid for smooth curve rendering
> dense_x <- seq(min(plot_data_log$X), max(plot_data_log$X),
+ length.out = 200)
> prediction_grid <- data.frame(X = dense_x)
> # Extract 95% Confidence and Prediction Intervals in the log transformed
> # scale
> ci_log <- predict(log_model, newdata = prediction_grid,
+ interval = "confidence", level = 0.95)
> pi_log <- predict(log_model, newdata = prediction_grid,
+ interval = "prediction", level = 0.95)
> # 4. Construct the interval tracking data frame (Both Scales)
> interval_bands_log <- data.frame(
+ X = dense_x,
+ # Transformed scale metrics: ln(Y)
+ Log_Fitted = ci_log[, "fit"],
+ Log_CI_Lwr = ci_log[, "lwr"],
+ Log_CI_Upr = ci_log[, "upr"],
+ Log_PI_Lwr = pi_log[, "lwr"],
+ Log_PI_Upr = pi_log[, "upr"],
+ # Back-transformed original scale metrics via exponentiation: exp(ln(Y))
+ Orig_Fitted = exp(ci_log[, "fit"]),
+ Orig_CI_Lwr = exp(ci_log[, "lwr"]),
+ Orig_CI_Upr = exp(ci_log[, "upr"]),
+ Orig_PI_Lwr = exp(pi_log[, "lwr"]),
+ Orig_PI_Upr = exp(pi_log[, "upr"])
+ )
> # 5. Calculate exact high-precision target metrics at x0 = 1.25% for
> # comparison
> target_x0 <- data.frame(X = 1.25)
> target_pi <- predict(log_model, newdata = target_x0,
+ interval = "prediction", level = 0.95)
> orig_pi_lwr <- exp(target_pi[1, "lwr"])
> orig_pi_upr <- exp(target_pi[1, "upr"])
> # 6. Build the visualization mapping back to the original data scale (Fixed
> # Version)
> transformed_regression_plot <- ggplot() +
+ # Layer A: Plot the raw data points on their original scale
+ geom_point(data = plot_data_log, aes(x = X, y = Y), color = "black",
+ size = 2.5) +
+ # Layer B: Back-transformed 95% prediction intervals (creates curved,

```

```

+ # opening lines)
+ geom_line(data = interval_bands_log, aes(x = X, y = Orig_PI_Lwr),
+ linetype = "dashed", color = "black", linewidth = 0.8) +
+ geom_line(data = interval_bands_log, aes(x = X, y = Orig_PI_Upr),
+ linetype = "dashed", color = "black", linewidth = 0.8) +
+ # Layer C: Back-transformed shaded 95% confidence intervals
+ geom_ribbon(data = interval_bands_log,
+ aes(x = X, ymin = Orig_CI_Lwr, ymax = Orig_CI_Upr), alpha = 0.15,
+ fill = "black") +
+ # Layer D: Back-transformed non-linear fitted regression curve
+ geom_line(data = interval_bands_log, aes(x = X, y = Orig_Fitted),
+ linetype = "solid", color = "black", linewidth = 1.0) +
+ # Layer E: Example 6-5 comparative reference markers at x0 = 1.25%
+ geom_vline(xintercept = 1.25, linetype = "dotted", color = "black",
+ linewidth = 0.8) +
+ annotate("point", x = 1.25, y = orig_pi_lwr, color = "black", size = 3,
+ shape = 21, fill = "white", stroke = 1.5) +
+ annotate("point", x = 1.25, y = orig_pi_upr, color = "black", size = 3,
+ shape = 21, fill = "white", stroke = 1.5) +
+ annotate("text", x = 1.29, y = orig_pi_lwr,
+ label = paste("New Lwr =", signif(orig_pi_lwr, 4)), hjust = 0,
+ size = 3.5, fontface = "bold") +
+ annotate("text", x = 1.29, y = orig_pi_upr,
+ label = paste("New Upr =", signif(orig_pi_upr, 4)), hjust = 0,
+ size = 3.5, fontface = "bold") +
+ # Layer F: Labels and formatting
+ labs(
+ title = "Log-Transformed Regression Model Back-Transformed to Original
+ Scale",
+ subtitle = "Model Equation: ln(Y) = 1.638 + 1.157X | Shaded: 95% CI |
+ Dashed: 95% PI",
+ x = "Roadway Area (%)",
+ y = "Salt Concentration (mg/l)"
+ ) +
+ theme(
+ plot.title = element_text(face = "bold", size = 11, hjust = 0.5),
+ plot.subtitle = element_text(size = 9, hjust = 0.5, face = "italic"),
+ axis.title = element_text(face = "bold", size = 11)
+ )
> # 7. Render the final plot window safely without object errors
> print(transformed_regression_plot)
> # 8. Print transformed summary parameter details for logging
> cat("--- Log-Transformed Regression Summary Parameter Details ---\n")
--- Log-Transformed Regression Summary Parameter Details ---
> cat(sprintf("Intercept (beta0_hat in log scale): %s\n",
+ signif(log_summary$coefficients[1,1], 4)))
Intercept (beta0_hat in log scale): 1.823
> cat(sprintf("Slope (beta1_hat in log scale): %s\n",
+ signif(log_summary$coefficients[2,1], 4)))
Slope (beta1_hat in log scale): 1.093
> cat(sprintf("Log-Model Fit Parameter (R^2): %s\n",
+ signif(log_summary$r.squared, 4)))
Log-Model Fit Parameter (R^2): 0.8145
> # resid(lm.Watershed)
> opar <- par(mfrow = c(2, 2), oma = c(0, .3, 1.3, 0))
> plot(lm.Watershed); par(opar)
> qqplot(lm.Watershed, reps = 10000)
[1] 10 12
> residualPlots(lm.Watershed, smooth = T, id = T)
      Test stat Pr(>|Test stat|)
roadway_area    -0.469         0.6450
Tukey test      -0.469         0.6391
> ols_test_breusch_pagan(lm.Watershed)

Breusch Pagan Test for Heteroskedasticity
-----
Ho: the variance is constant
Ha: the variance is not constant

Data
-----
Response : salt_concentration
Variables: fitted values of salt_concentration

```

## Test Summary

```
-----
DF          =      1
Chi2        =      0.1246598
Prob > Chi2 =      0.7240346
> ols_test_f(lm.Watershed)
```

## F Test for Heteroskedasticity

```
-----
Ho: Variance is homogenous
Ha: Variance is not homogenous
```

Variables: fitted values of salt\_concentration

## Test Summary

```
-----
Num DF      =      1
Den DF      =     18
F           =      0.09858949
Prob > F    =      0.7571388
> ols_test_normality(lm.Watershed)
```

```
-----
Test                Statistic      pvalue
-----
Shapiro-Wilk        0.9765         0.8808
Kolmogorov-Smirnov  0.1184         0.9114
Cramer-von Mises    1.1149         0.0011
Anderson-Darling    0.2619         0.6671
-----
```

```
> ols_test_outlier(lm.Watershed)
      studentized residual unadjusted p_val bonferroni p_val
10      -2.584649         0.01928221      0.3856441
> outlierTest(lm.Watershed)
No Studentized residuals with Bonferroni p < 0.05
Largest |rstudent|:
      rstudent unadjusted p-value Bonferroni p
10 -2.584649         0.019282      0.38564
> ols_test_score(lm.Watershed)
```

## Score Test for Heteroskedasticity

```
-----
Ho: Variance is homogenous
Ha: Variance is not homogenous
```

Variables: fitted values of salt\_concentration

## Test Summary

```
-----
DF          =      1
Chi2        =      0.1089472
Prob > Chi2 =      0.741346
> influence.measures(lm.Watershed)
```

Influence measures of

lm(formula = salt\_concentration ~ roadway\_area, data = Watershed) :

|    | dfb.1     | dfb.rdw  | dffit    | cov.r | cook.d   | hat    | inf |
|----|-----------|----------|----------|-------|----------|--------|-----|
| 1  | -0.597151 | 0.49806  | -0.60111 | 1.079 | 1.72e-01 | 0.1595 |     |
| 2  | 0.161929  | -0.13717 | 0.16254  | 1.337 | 1.39e-02 | 0.1738 | *   |
| 3  | 0.186243  | -0.11184 | 0.21929  | 1.114 | 2.45e-02 | 0.0676 |     |
| 4  | 0.127755  | -0.09440 | 0.13420  | 1.221 | 9.44e-03 | 0.0990 |     |
| 5  | -0.033137 | 0.01336  | -0.04804 | 1.180 | 1.22e-03 | 0.0542 |     |
| 6  | 0.006743  | -0.00311 | 0.00918  | 1.188 | 4.46e-05 | 0.0565 |     |
| 7  | 0.155294  | -0.08148 | 0.19752  | 1.112 | 1.99e-02 | 0.0603 |     |
| 8  | 0.155966  | -0.10790 | 0.16939  | 1.180 | 1.49e-02 | 0.0841 |     |
| 9  | -0.026489 | 0.00745  | -0.04374 | 1.177 | 1.01e-03 | 0.0515 |     |
| 10 | -0.552598 | 0.31229  | -0.67399 | 0.617 | 1.73e-01 | 0.0637 | *   |
| 11 | -0.054158 | 0.01007  | -0.09855 | 1.156 | 5.09e-03 | 0.0505 |     |
| 12 | 0.288452  | -0.01923 | 0.58865  | 0.614 | 1.32e-01 | 0.0501 | *   |
| 13 | -0.128163 | 0.02383  | -0.23322 | 1.051 | 2.72e-02 | 0.0505 |     |
| 14 | 0.177760  | -0.07526 | 0.25210  | 1.047 | 3.16e-02 | 0.0549 |     |
| 15 | -0.008083 | 0.01712  | 0.02304  | 1.261 | 2.81e-04 | 0.1117 |     |
| 16 | 0.000176  | -0.01372 | -0.02941 | 1.196 | 4.57e-04 | 0.0639 |     |
| 17 | -0.080601 | 0.13360  | 0.15688  | 1.353 | 1.29e-02 | 0.1820 | *   |
| 18 | 0.006139  | -0.12310 | -0.25513 | 1.078 | 3.27e-02 | 0.0652 |     |



```

19 0.482328 -0.71828 -0.79296 1.294 3.04e-01 0.2786
20 -0.304948 0.47724 0.54197 1.283 1.47e-01 0.2226
> summary(influence.measures(lm.Watershed))
Potentially influential observations of
lm(formula = salt_concentration ~ roadway_area, data = Watershed) :

    dfb.1_ dfb.rdw_ dffit cov.r    cook.d hat
2   0.16_ -0.14_    0.16 1.34_* 0.01 0.17
10 -0.55_ 0.31_   -0.67 0.62_* 0.17 0.06
12 0.29_ -0.02_    0.59 0.61_* 0.13 0.05
17 -0.08_ 0.13_    0.16 1.35_* 0.01 0.18
> case(lm.Watershed)
      e          h          si      sta.res      stu.res      dffit
1 -2.21041306 0.15950451 1.747319 -1.34645834 -1.37985524 -0.419479754
2 0.59145363 0.17375800 1.835802 0.36337433 0.35443878 0.124382198
3 1.42185344 0.06757603 1.807632 0.82230948 0.81458815 0.103046706
4 0.70478685 0.09897621 1.833752 0.41464539 0.40490128 0.077419857
5 -0.35921328 0.05418887 1.840394 -0.20627061 -0.20069634 -0.020580601
6 0.06718673 0.05646093 1.842496 0.03862698 0.03754023 0.004020422
7 1.36905342 0.06025314 1.810456 0.78868236 0.78005808 0.087778712
8 0.97652015 0.08413972 1.825877 0.56984111 0.55884977 0.089712523
9 -0.33654663 0.05149182 1.840666 -0.19297982 -0.18773698 -0.018270163
10 -3.90454657 0.06366940 1.561185 -2.25342535 -2.58464870 -0.265504668
11 -0.76294665 0.05052742 1.832761 -0.43726039 -0.42721572 -0.040601202
12 3.91065334 0.05005340 1.564632 2.24071644 2.56441063 0.206055245
13 -1.76294665 0.05052742 1.789559 -1.01038091 -1.01100167 -0.093817507
14 1.81625339 0.05489174 1.785989 1.04333274 1.04606046 0.105487706
15 0.11278646 0.11172589 1.842344 0.06682987 0.06495502 0.014186125
16 -0.20054677 0.06391459 1.841887 -0.11575642 -0.11253692 -0.013693049
17 0.55251970 0.18196901 1.836606 0.34115368 0.33261887 0.122906665
18 -1.67601344 0.06517321 1.793967 -0.96805292 -0.96626772 -0.116846434
19 -1.90774707 0.27858326 1.760203 -1.25433882 -1.27604219 -0.736698176
20 1.59785299 0.22261543 1.789380 1.01205948 1.01278252 0.457568544
      dffits      cook (Intercept) roadway_area
1 -0.601107727 1.720257e-01 -0.5971513834 0.498060187
2 0.162539901 1.388405e-02 0.1619286909 -0.137174766
3 0.219294494 2.450303e-02 0.1862433136 -0.111838628
4 0.134198120 9.443180e-03 0.1277554853 -0.094400352
5 -0.048038815 1.218850e-03 -0.0331365544 0.013356284
6 0.009183136 4.464159e-05 0.0067425687 -0.003106451
7 0.197520123 1.994082e-02 0.1552936910 -0.081479803
8 0.169387361 1.491590e-02 0.1559656566 -0.107897271
9 -0.043741978 1.010860e-03 -0.0264893062 0.007445395
10 -0.673988210 1.726466e-01 -0.5525978586 0.312292583
11 -0.098552940 5.087390e-03 -0.0541575000 0.010068979
12 0.588646743 1.322751e-01 0.2884517128 -0.019226168
13 -0.233224533 2.716345e-02 -0.1281631745 0.023828136
14 0.252098053 3.161121e-02 0.1777600395 -0.075257111
15 0.023036482 2.808783e-04 -0.0080832412 0.017122727
16 -0.029406090 4.574511e-04 0.0001758467 -0.013720580
17 0.156877520 1.294487e-02 -0.0806005241 0.133597391
18 -0.255132894 3.266677e-02 0.0061389872 -0.123103567
19 -0.792956626 3.037865e-01 0.4823283677 -0.718280598
20 0.541970774 1.466565e-01 -0.3049475959 0.477241325
attr(,"class")
[1] "case" "matrix" "array"
> cooks.distance(lm.Watershed)
      1          2          3          4          5          6
1.720257e-01 1.388405e-02 2.450303e-02 9.443180e-03 1.218850e-03 4.464159e-05
      7          8          9         10         11         12
1.994082e-02 1.491590e-02 1.010860e-03 1.726466e-01 5.087390e-03 1.322751e-01
      13         14         15         16         17         18
2.716345e-02 3.161121e-02 2.808783e-04 4.574511e-04 1.294487e-02 3.266677e-02
      19         20
3.037865e-01 1.466565e-01
> covratio(lm.Watershed)
      1          2          3          4          5          6          7          8
1.0787037 1.3370412 1.1137184 1.2206033 1.1797424 1.1879968 1.1119618 1.1803358
      9         10         11         12         13         14         15         16
1.1770832 0.6170742 1.1558173 0.6136181 1.0506320 1.0470867 1.2614923 1.1958724
      17         18         19         20
1.3528292 1.0776441 1.2942349 1.2826951
> dfbetas(lm.Watershed)
      (Intercept) roadway_area

```

```

1 -0.5971513834 0.498060187
2 0.1619286909 -0.137174766
3 0.1862433136 -0.111838628
4 0.1277554853 -0.094400352
5 -0.0331365544 0.013356284
6 0.0067425687 -0.003106451
7 0.1552936910 -0.081479803
8 0.1559656566 -0.107897271
9 -0.0264893062 0.007445395
10 -0.5525978586 0.312292583
11 -0.0541575000 0.010068979
12 0.2884517128 -0.019226168
13 -0.1281631745 0.023828136
14 0.1777600395 -0.075257111
15 -0.0080832412 0.017122727
16 0.0001758467 -0.013720580
17 -0.0806005241 0.133597391
18 0.0061389872 -0.123103567
19 0.4823283677 -0.718280598
20 -0.3049475959 0.477241325
> dffits(lm.Watershed)
      1      2      3      4      5      6
-0.601107727 0.162539901 0.219294494 0.134198120 -0.048038815 0.009183136
      7      8      9     10     11     12
 0.197520123 0.169387361 -0.043741978 -0.673988210 -0.098552940 0.588646743
     13     14     15     16     17     18
-0.233224533 0.252098053 0.023036482 -0.029406090 0.156877520 -0.255132894
     19     20
-0.792956626 0.541970774
> hatvalues(lm.Watershed)
      1      2      3      4      5      6      7
0.15950451 0.17375800 0.06757603 0.09897621 0.05418887 0.05646093 0.06025314
      8      9     10     11     12     13     14
0.08413972 0.05149182 0.06366940 0.05052742 0.05005340 0.05052742 0.05489174
     15     16     17     18     19     20
0.11172589 0.06391459 0.18196901 0.06517321 0.27858326 0.22261543
> ols_plot_cooksd_chart(lm.Watershed)
> ols_plot_dfbetas(lm.Watershed)
> ols_plot_dffits(lm.Watershed)
> ols_plot_diagnostics(lm.Watershed)
> ols_plot_hadi(lm.Watershed)
> ols_plot_resid_box(lm.Watershed)
> ols_plot_resid_fit(lm.Watershed)
> ols_plot_resid_fit_spread(lm.Watershed)
> ols_plot_resid_hist(lm.Watershed)
> ols_plot_resid_lev(lm.Watershed)
> ols_plot_resid_pot(lm.Watershed)
> ols_plot_resid_qq(lm.Watershed)
> ols_plot_resid_regressor(lm.Watershed)
> ols_plot_resid_spread(lm.Watershed)
> ols_plot_resid_stand(lm.Watershed)
> ols_plot_resid_stud(lm.Watershed)
> ols_plot_resid_stud_fit(lm.Watershed)
> salt_concentration0 <- salt_concentration + c(rep(0, 11), 8, rep(0, 8))
> (Watershed0 <- data.frame(salt_concentration0, roadway_area)
  salt_concentration0 roadway_area
1          3.8         0.19
2          5.9         0.15
3         14.1         0.57
4         10.4         0.40
5         14.6         0.70
6         14.5         0.67
7         15.1         0.63
8         11.9         0.47
9         15.5         0.75
10          9.3         0.60
11         15.6         0.78
12         28.8         0.81
13         14.6         0.78
14         16.6         0.69
15         25.6         1.30
16         20.9         1.05
17         29.9         1.52
18         19.6         1.06

```

```

19          31.3          1.74
20          32.7          1.62
> cor(salt_concentration0, roadway_area)
[1] 0.9281614
> spm(Watershed0)
> sp(roadway_area, salt_concentration0, "xy", T, T, T, T,
+ xlab = "Roadway area (%)", ylab = "Salt conc. (mg/l)")
[1] 12 19
> regressedplot(roadway_area, salt_concentration0, "square",
+ xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
> ols_regress(salt_concentration0 ~ roadway_area, Watershed0)
Model Summary

```

|                |       |           |         |
|----------------|-------|-----------|---------|
| R              | 0.928 | RMSE      | 3.009   |
| R-Squared      | 0.861 | MSE       | 9.054   |
| Adj. R-Squared | 0.854 | Coef. Var | 18.088  |
| Pred R-Squared | 0.840 | AIC       | 106.822 |
| MAE            | 1.732 | SBC       | 109.809 |

```

RMSE: Root Mean Square Error
MSE: Mean Square Error
MAE: Mean Absolute Error
AIC: Akaike Information Criteria
SBC: Schwarz Bayesian Criteria

```

## ANOVA

|            | Sum of Squares | DF | Mean Square | F       | Sig.   |
|------------|----------------|----|-------------|---------|--------|
| Regression | 1126.222       | 1  | 1126.222    | 111.948 | 0.0000 |
| Residual   | 181.083        | 18 | 10.060      |         |        |
| Total      | 1307.306       | 19 |             |         |        |

## Parameter Estimates

| model        | Beta   | Std. Error | Std. Beta | t      | Sig.  | lower  | upper  |
|--------------|--------|------------|-----------|--------|-------|--------|--------|
| (Intercept)  | 3.102  | 1.537      |           | 2.017  | 0.059 | -0.128 | 6.332  |
| roadway_area | 17.516 | 1.656      | 0.928     | 10.581 | 0.000 | 14.038 | 20.994 |

```
> (lm.Watershed0 <- lm(salt_concentration0 ~ roadway_area, Watershed0))
```

```

Call:
lm(formula = salt_concentration0 ~ roadway_area, data = Watershed0)

```

```

Coefficients:
(Intercept)  roadway_area
      3.102         17.516

```

```

> opar <- par(mfrow = c(2, 2), oma = c(0, .3, 1.3, 0))
> plot(lm.Watershed0); par(opar)
> summary(lm.Watershed0)

```

```

Call:
lm(formula = salt_concentration0 ~ roadway_area, data = Watershed0)

```

```

Residuals:
    Min       1Q   Median       3Q      Max
-4.3114 -1.3904 -0.3051  0.6651 11.5102

```

```

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)    3.102     1.537   2.017  0.0588 .
roadway_area   17.516     1.656  10.581 3.72e-09 ***
---

```

```

Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

Residual standard error: 3.172 on 18 degrees of freedom
Multiple R-squared:  0.8615,    Adjusted R-squared:  0.8538
F-statistic: 111.9 on 1 and 18 DF,  p-value: 3.722e-09

```

```
> anova(lm.Watershed0)
```

## Analysis of Variance Table

Response: salt\_concentrationO

|              | Df | Sum Sq  | Mean Sq | F value | Pr(>F)        |
|--------------|----|---------|---------|---------|---------------|
| roadway_area | 1  | 1126.22 | 1126.22 | 111.95  | 3.722e-09 *** |
| Residuals    | 18 | 181.08  | 10.06   |         |               |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

&gt; lht(lm.WatershedO, "roadway\_area")

Linear hypothesis test:

roadway\_area = 0

Model 1: restricted model

Model 2: salt\_concentrationO ~ roadway\_area

|   | Res.Df | RSS     | Df | Sum of Sq | F      | Pr(>F)        |
|---|--------|---------|----|-----------|--------|---------------|
| 1 | 19     | 1307.31 |    |           |        |               |
| 2 | 18     | 181.08  | 1  | 1126.2    | 111.95 | 3.722e-09 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

&gt; confint(lm.WatershedO)

|              | 2.5 %      | 97.5 %    |
|--------------|------------|-----------|
| (Intercept)  | -0.1284543 | 6.331831  |
| roadway_area | 14.0380747 | 20.994235 |

&gt; ci.plot(lm.WatershedO, main = "95% confidence and prediction intervals",

+ xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")

&gt; residualPlots(lm.WatershedO, smooth = T, id = T)

|              | Test stat | Pr(> Test stat ) |
|--------------|-----------|------------------|
| roadway_area | -0.7476   | 0.4649           |
| Tukey test   | -0.7476   | 0.4547           |

&gt; ols\_test\_breusch\_pagan(lm.WatershedO)

Breusch Pagan Test for Heteroskedasticity

-----

Ho: the variance is constant

Ha: the variance is not constant

|            | Data                                 |
|------------|--------------------------------------|
| Response   | : salt_concentrationO                |
| Variables: | fitted values of salt_concentrationO |

-----

Response : salt\_concentrationO

Variables: fitted values of salt\_concentrationO

| Test Summary |              |
|--------------|--------------|
| DF           | = 1          |
| Chi2         | = 0.03486086 |
| Prob > Chi2  | = 0.8518875  |

-----

&gt; ols\_test\_f(lm.WatershedO)

F Test for Heteroskedasticity

-----

Ho: Variance is homogenous

Ha: Variance is not homogenous

Variables: fitted values of salt\_concentrationO

| Test Summary |               |
|--------------|---------------|
| Num DF       | = 1           |
| Den DF       | = 18          |
| F            | = 0.006281479 |
| Prob > F     | = 0.9377037   |

-----

&gt; ols\_test\_normality(lm.WatershedO)

| Test               | Statistic | pvalue |
|--------------------|-----------|--------|
| Shapiro-Wilk       | 0.7045    | 0.0000 |
| Kolmogorov-Smirnov | 0.2737    | 0.0816 |
| Cramer-von Mises   | 1.738     | 0.0000 |
| Anderson-Darling   | 1.7596    | 1e-04  |

-----

-----

&gt; ols\_test\_outlier(lm.WatershedO)

|  | studentized_residual | unadjusted_p_val | bonferroni_p_val |
|--|----------------------|------------------|------------------|
|  |                      |                  |                  |

```

12          7.54783      7.985395e-07      1.597079e-05
> outlierTest(lm.Watershed0)
      rstudent unadjusted p-value Bonferroni p
12  7.54783      7.9854e-07      1.5971e-05
> ols_test_score(lm.Watershed0)

Score Test for Heteroskedasticity
-----
Ho: Variance is homogenous
Ha: Variance is not homogenous

Variables: fitted values of salt_concentration0

Test Summary
-----
DF          =      1
Chi2        =      0.006976987
Prob > Chi2  =      0.9334314
> qqplot(lm.Watershed0, reps = 10000)
[1] 10 12
> influence.measures(lm.Watershed0)
Influence measures of
      lm(formula = salt_concentration0 ~ roadway_area, data = Watershed0) :

      dfb.1_ dfb.rdw_ dffit cov.r cook.d hat inf
1  -0.389298  0.32470 -0.3919 1.2154 0.077606 0.1595
2   0.026319 -0.02230  0.0264 1.3563 0.000369 0.1738 *
3   0.073796 -0.04431  0.0869 1.1878 0.003973 0.0676
4   0.029732 -0.02197  0.0312 1.2430 0.000516 0.0990
5  -0.039757  0.01602 -0.0576 1.1773 0.001753 0.0542
6  -0.019128  0.00881 -0.0261 1.1866 0.000359 0.0565
7   0.060769 -0.03188  0.0773 1.1800 0.003146 0.0603
8   0.050597 -0.03500  0.0550 1.2194 0.001596 0.0841
9  -0.032848  0.00923 -0.0542 1.1745 0.001553 0.0515
10 -0.309321  0.17481 -0.3773 0.9492 0.067092 0.0637
11 -0.046595  0.00866 -0.0848 1.1622 0.003776 0.0505
12  0.849000 -0.05659  1.7326 0.0623 0.365230 0.0501 *
13 -0.087472  0.01626 -0.1592 1.1173 0.013048 0.0505
14  0.076077 -0.03221  0.1079 1.1587 0.006091 0.0549
15  0.011035 -0.02337 -0.0314 1.2610 0.000523 0.1117
16  0.000294 -0.02295 -0.0492 1.1927 0.001278 0.0639
17 -0.014265  0.02364  0.0278 1.3699 0.000408 0.1820 *
18  0.004219 -0.08460 -0.1753 1.1394 0.015864 0.0652
19  0.317236 -0.47243 -0.5215 1.4328 0.138274 0.2786 *
20 -0.128562  0.20120  0.2285 1.4117 0.027346 0.2226 *
> summary(influence.measures(lm.Watershed0))
Potentially influential observations of
      lm(formula = salt_concentration0 ~ roadway_area, data = Watershed0) :

      dfb.1_ dfb.rdw_ dffit cov.r cook.d hat
2   0.03 -0.02  0.03  1.36_*  0.00  0.17
12  0.85 -0.06  1.73_*  0.06_*  0.37  0.05
17 -0.01  0.02  0.03  1.37_*  0.00  0.18
19  0.32 -0.47 -0.52  1.43_*  0.14  0.28
20 -0.13  0.20  0.23  1.41_*  0.03  0.22
> case(lm.Watershed0)
      e h si sta.res stu.res dffit
1 -2.6297577 0.15950451 3.188723 -0.90436782 -0.89956221 -0.49906062
2  0.1708885 0.17375800 3.263415  0.05927291  0.05760854  0.03593771
3  1.0141034 0.06757603 3.253779  0.33111021  0.32276574  0.07349563
4  0.2918497 0.09897621 3.262882  0.09693670  0.09423014  0.03205929
5 -0.7629968 0.05418887 3.258182 -0.24735321 -0.24079370 -0.04371479
6 -0.3375121 0.05646093 3.262646 -0.10954851 -0.10649753 -0.02019656
7  0.9631341 0.06025314 3.254826  0.31324083  0.30524848  0.06175264
8  0.5657189 0.08413972 3.260583  0.18637306  0.18129707  0.05197237
9 -0.7388045 0.05149182 3.258544 -0.23916965 -0.23280133 -0.04010760
10 -4.3113813 0.06366940 3.079642 -1.40474950 -1.44677694 -0.29316896
11 -1.1642892 0.05052742 3.250842 -0.37671829 -0.36755620 -0.06195917
12 11.5102262 0.05005340 1.564632  3.72332822  7.54782992  0.60648241
13 -2.1642892 0.05052742 3.218968 -0.70027905 -0.69001315 -0.11517547
14  1.4121648 0.05489174 3.244663  0.45797488  0.44768751  0.08201830
15 -0.2726898 0.11172589 3.262979 -0.09122050 -0.08867089 -0.03429855
16 -0.5936510 0.06391459 3.260339 -0.19345081 -0.18819613 -0.04053365
17  0.1737561 0.18196901 3.263401  0.06056925  0.05886873  0.03865162

```

```

18 -2.0688126 0.06517321 3.222211 -0.67460979 -0.66405130 -0.14423117
19 -2.2797980 0.27858326 3.198150 -0.84625320 -0.83927536 -0.88036987
20 1.2221406 0.22261543 3.246373 0.43701895 0.42697730 0.34997781
      dffits      cook      (Intercept) roadway_area
1 -0.39187719 0.0776064427 -0.3892979528 0.324697919
2 0.02641835 0.0003694197 0.0263190033 -0.022295636
3 0.08689145 0.0039727793 0.0737955252 -0.044314022
4 0.03123109 0.0005161086 0.0297317347 -0.021969203
5 -0.05763655 0.0017527129 -0.0397569464 0.016024753
6 -0.02605155 0.0003590634 -0.0191279283 0.008812661
7 0.07729260 0.0031455425 0.0607687605 -0.031884274
8 0.05495114 0.0015955415 0.0505969897 -0.035003072
9 -0.05424179 0.0015526706 -0.0328477935 0.009232586
10 -0.37727007 0.0670917821 -0.3093208917 0.174808170
11 -0.08479029 0.0037761405 -0.0465945511 0.008662873
12 1.73256399 0.3652304713 0.8489999409 -0.056588383
13 -0.15917679 0.0130483921 -0.0874719391 0.016262809
14 0.10789161 0.0060908616 0.0760768170 -0.032208147
15 -0.03144738 0.0005233133 0.0110345310 -0.023374442
16 -0.04917597 0.0012776021 0.0002940694 -0.022945004
17 0.02776505 0.0004080394 -0.0142651280 0.023644808
18 -0.17533581 0.0158640199 0.0042189161 -0.084600863
19 -0.52154150 0.1382736596 0.3172358402 -0.472425767
20 0.22848856 0.0273457529 -0.1285623506 0.201199378
attr(,"class")
[1] "case" "matrix" "array"
> cooks.distance(lm.WatershedO)
      1      2      3      4      5      6
0.0776064427 0.0003694197 0.0039727793 0.0005161086 0.0017527129 0.0003590634
      7      8      9     10     11     12
0.0031455425 0.0015955415 0.0015526706 0.0670917821 0.0037761405 0.3652304713
      13     14     15     16     17     18
0.0130483921 0.0060908616 0.0005233133 0.0012776021 0.0004080394 0.0158640199
      19     20
0.1382736596 0.0273457529
> covratio(lm.WatershedO)
      1      2      3      4      5      6      7
1.21540272 1.35634555 1.18775586 1.24296057 1.17729502 1.18660994 1.18001782
      8      9     10     11     12     13     14
1.21938316 1.17446859 0.94920499 1.16222288 0.06233582 1.11730731 1.15873771
      15     16     17     18     19     20
1.26095194 1.19268005 1.36993626 1.13939126 1.43283835 1.41171169
> dfbetas(lm.WatershedO)
      (Intercept) roadway_area
1 -0.3892979528 0.324697919
2 0.0263190033 -0.022295636
3 0.0737955252 -0.044314022
4 0.0297317347 -0.021969203
5 -0.0397569464 0.016024753
6 -0.0191279283 0.008812661
7 0.0607687605 -0.031884274
8 0.0505969897 -0.035003072
9 -0.0328477935 0.009232586
10 -0.3093208917 0.174808170
11 -0.0465945511 0.008662873
12 0.8489999409 -0.056588383
13 -0.0874719391 0.016262809
14 0.0760768170 -0.032208147
15 0.0110345310 -0.023374442
16 0.0002940694 -0.022945004
17 -0.0142651280 0.023644808
18 0.0042189161 -0.084600863
19 0.3172358402 -0.472425767
20 -0.1285623506 0.201199378
> dffits(lm.WatershedO)
      1      2      3      4      5      6
-0.39187719 0.02641835 0.08689145 0.03123109 -0.05763655 -0.02605155
      7      8      9     10     11     12
0.07729260 0.05495114 -0.05424179 -0.37727007 -0.08479029 1.73256399
      13     14     15     16     17     18
-0.15917679 0.10789161 -0.03144738 -0.04917597 0.02776505 -0.17533581
      19     20
-0.52154150 0.22848856
> hatvalues(lm.WatershedO)

```

```

      1      2      3      4      5      6      7
0.15950451 0.17375800 0.06757603 0.09897621 0.05418887 0.05646093 0.06025314
      8      9     10     11     12     13     14
0.08413972 0.05149182 0.06366940 0.05052742 0.05005340 0.05052742 0.05489174
     15     16     17     18     19     20
0.11172589 0.06391459 0.18196901 0.06517321 0.27858326 0.22261543
> ols_plot_cooksd_chart(lm.WatershedO)
> ols_plot_dfbetas(lm.WatershedO)
> ols_plot_dffits(lm.WatershedO)
> ols_plot_diagnostics(lm.WatershedO)
> ols_plot_hadi(lm.WatershedO)
> ols_plot_resid_box(lm.WatershedO)
> ols_plot_resid_fit(lm.WatershedO)
> ols_plot_resid_fit_spread(lm.WatershedO)
> ols_plot_resid_hist(lm.WatershedO)
> ols_plot_resid_lev(lm.WatershedO)
> ols_plot_resid_pot(lm.WatershedO)
> ols_plot_resid_qq(lm.WatershedO)
> ols_plot_resid_regressor(lm.WatershedO)
> ols_plot_resid_spread(lm.WatershedO)
> ols_plot_resid_stand(lm.WatershedO)
> ols_plot_resid_stud(lm.WatershedO)
> ols_plot_resid_stud_fit(lm.WatershedO)
> # To analyze this data set, one possible way is first to remove (x12, y12)
> # and then to analyze the remaining data set in Table 6-1.
> # Figure 6-16 Effect of an influential observation.
> salt_concentrationI <- salt_concentration + c(rep(0, 18), 30, 0)
> (WatershedI <- data.frame(salt_concentrationI, roadway_area))
  salt_concentrationI roadway_area
1                3.8          0.19
2                 5.9          0.15
3                14.1          0.57
4                10.4          0.40
5                14.6          0.70
6                14.5          0.67
7                15.1          0.63
8                11.9          0.47
9                15.5          0.75
10               9.3          0.60
11               15.6          0.78
12               20.8          0.81
13               14.6          0.78
14               16.6          0.69
15               25.6          1.30
16               20.9          1.05
17               29.9          1.52
18               19.6          1.06
19               61.3          1.74
20               32.7          1.62
> cor(salt_concentrationI, roadway_area)
[1] 0.8917228
> spm(WatershedI)
> sp(roadway_area, salt_concentrationI, "xy", T, T, T, T,
+ xlab = "Roadway area (%)", ylab = "Salt conc. (mg/l)")
[1] 19 20
> regrresidplot(roadway_area, salt_concentrationI, "square",
+ xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
> ols_regress(salt_concentrationI ~ roadway_area, WatershedI)

      Model Summary
-----
R                0.892      RMSE                5.443
R-Squared        0.795      MSE                 29.626
Adj. R-Squared   0.784      Coef. Var          30.788
Pred R-Squared   0.647      AIC                130.531
MAE              3.639      SBC                133.518
-----
RMSE: Root Mean Square Error
MSE: Mean Square Error
MAE: Mean Absolute Error
AIC: Akaike Information Criteria
SBC: Schwarz Bayesian Criteria

      ANOVA
-----

```

|            | Sum of Squares | DF | Mean Square | F      | Sig.   |
|------------|----------------|----|-------------|--------|--------|
| Regression | 2300.239       | 1  | 2300.239    | 69.878 | 0.0000 |
| Residual   | 592.526        | 18 | 32.918      |        |        |
| Total      | 2892.766       | 19 |             |        |        |

| Parameter Estimates |        |            |           |        |       |        |        |
|---------------------|--------|------------|-----------|--------|-------|--------|--------|
| model               | Beta   | Std. Error | Std. Beta | t      | Sig   | lower  | upper  |
| (Intercept)         | -1.992 | 2.781      |           | -0.716 | 0.483 | -7.835 | 3.851  |
| roadway_area        | 25.033 | 2.995      | 0.892     | 8.359  | 0.000 | 18.742 | 31.325 |

```
> (lm.WatershedI <- lm(salt_concentrationI ~ roadway_area, WatershedI))
```

Call:

```
lm(formula = salt_concentrationI ~ roadway_area, data = WatershedI)
```

Coefficients:

```
(Intercept)  roadway_area
-1.992      25.033
```

```
> opar <- par(mfrow = c(2, 2), oma = c(0, .3, 1.3, 0))
```

```
> plot(lm.WatershedI); par(opar)
```

```
> summary(lm.WatershedI)
```

Call:

```
lm(formula = salt_concentrationI ~ roadway_area, data = WatershedI)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-6.1580 -3.4762 -0.6054  1.8992 19.7348
```

Coefficients:

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   -1.992     2.781   -0.716    0.483
roadway_area   25.033     2.995    8.359 1.3e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 5.737 on 18 degrees of freedom

Multiple R-squared: 0.7952, Adjusted R-squared: 0.7838

F-statistic: 69.88 on 1 and 18 DF, p-value: 1.304e-07

```
> anova(lm.WatershedI)
```

Analysis of Variance Table

Response: salt\_concentrationI

```
      Df Sum Sq Mean Sq F value    Pr(>F)
roadway_area  1 2300.24  2300.24   69.878 1.304e-07 ***
Residuals    18  592.53    32.92
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
> lht(lm.WatershedI, "roadway_area")
```

Linear hypothesis test:

```
roadway_area = 0
```

Model 1: restricted model

Model 2: salt\_concentrationI ~ roadway\_area

```
      Res.Df    RSS Df Sum of Sq      F      Pr(>F)
1         19 2892.77
2         18  592.53   1    2300.2 69.878 1.304e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
> confint(lm.WatershedI)
```

```
              2.5 %      97.5 %
(Intercept) -7.835214  3.850799
roadway_area 18.741517 31.324520
```

```
> ci.plot(lm.WatershedI, main = "95% confidence and prediction intervals",
+ xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
```



```

> residualPlots(lm.WatershedI, smooth = T, id = T)
      Test stat Pr(>|Test stat|)
roadway_area      2.3577      0.03063 *
Tukey test       2.3577      0.01839 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> (lm.Watershed2I <- lm(salt_concentrationI ~ roadway_area +
+ I(roadway_area^2), WatershedI))

Call:
lm(formula = salt_concentrationI ~ roadway_area + I(roadway_area^2),
    data = WatershedI)

Coefficients:
      (Intercept)      roadway_area      I(roadway_area^2)
          7.40325           0.04454           12.97962

> opar <- par(mfrow = c(2, 2), oma = c(0, .3, 1.3, 0))
> plot(lm.Watershed2I); par(opar)
> summary(lm.Watershed2I)

Call:
lm(formula = salt_concentrationI ~ roadway_area + I(roadway_area^2),
    data = WatershedI)

Residuals:
      Min       1Q   Median       3Q      Max
-8.8391 -2.5264  0.5138  1.8200 14.5222

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)      7.40325      4.69601   1.576   0.1333
roadway_area      0.04454     10.93116   0.004   0.9968
I(roadway_area^2) 12.97962      5.50527   2.358   0.0306 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.125 on 17 degrees of freedom
Multiple R-squared:  0.8456,    Adjusted R-squared:  0.8275
F-statistic: 46.57 on 2 and 17 DF,  p-value: 1.266e-07

> anova(lm.Watershed2I)
Analysis of Variance Table

Response: salt_concentrationI
      Df Sum Sq Mean Sq F value    Pr(>F)
roadway_area      1 2300.24  2300.24  87.5745 4.057e-08 ***
I(roadway_area^2)  1  146.00   146.00   5.5586  0.03063 *
Residuals       17  446.52    26.27
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.Watershed2I, "I(roadway_area^2)")

Linear hypothesis test:
I(roadway_area^2) = 0

Model 1: restricted model
Model 2: salt_concentrationI ~ roadway_area + I(roadway_area^2)

      Res.Df    RSS Df Sum of Sq      F    Pr(>F)
1         18 592.53
2         17 446.52   1         146  5.5586 0.03063 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.Watershed2I, c("roadway_area", "I(roadway_area^2)"))

Linear hypothesis test:
roadway_area = 0
I(roadway_area^2) = 0

Model 1: restricted model
Model 2: salt_concentrationI ~ roadway_area + I(roadway_area^2)

      Res.Df    RSS Df Sum of Sq      F    Pr(>F)

```

```

1      19 2892.77
2      17 446.52  2      2446.2 46.566 1.266e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> ols_test_breusch_pagan(lm.WatershedI)

Breusch Pagan Test for Heteroskedasticity
-----
Ho: the variance is constant
Ha: the variance is not constant

Data
-----
Response : salt_concentrationI
Variables: fitted values of salt_concentrationI

Test Summary
-----
DF          =      1
Chi2         =    25.94769
Prob > Chi2  =   3.507942e-07
> ols_test_f(lm.WatershedI)

F Test for Heteroskedasticity
-----
Ho: Variance is homogenous
Ha: Variance is not homogenous

Variables: fitted values of salt_concentrationI

Test Summary
-----
Num DF      =      1
Den DF      =     18
F           =    8.795975
Prob > F    =   0.008278079
> ols_test_normality(lm.WatershedI)
-----

```

|  | Test               | Statistic | pvalue |
|--|--------------------|-----------|--------|
|  | Shapiro-Wilk       | 0.7702    | 3e-04  |
|  | Kolmogorov-Smirnov | 0.2262    | 0.2215 |
|  | Cramer-von Mises   | 1.7167    | 0.0000 |
|  | Anderson-Darling   | 1.153     | 0.0039 |

```

-----
> ols_test_outlier(lm.WatershedI)
      studentized_residual unadjusted_p_val bonferroni_p_val
19      13.20006      2.310508e-10      4.621015e-09
> outlierTest(lm.WatershedI)
      rstudent unadjusted p-value Bonferroni p
19 13.20006      2.3105e-10      4.621e-09
> ols_test_score(lm.WatershedI)

Score Test for Heteroskedasticity
-----
Ho: Variance is homogenous
Ha: Variance is not homogenous

Variables: fitted values of salt_concentrationI

Test Summary
-----
DF          =      1
Chi2         =    6.565146
Prob > Chi2  =   0.01039952
> qqplot(lm.WatershedI, reps = 10000)
[1] 17 19
> influence.measures(lm.WatershedI)
Influence measures of
      lm(formula = salt_concentrationI ~ roadway_area, data = WatershedI) :
      dfb.1 dfb.rdw dffit cov.r cook.d hat inf
1  0.082919 -0.06916 0.0835 1.3281 3.68e-03 0.1595
2  0.358542 -0.30373 0.3599 1.2637 6.62e-02 0.1738

```

```

3  0.073349 -0.04405  0.0864 1.1879 3.93e-03 0.0676
4  0.134661 -0.09950  0.1415 1.2180 1.05e-02 0.0990
5 -0.026790  0.01080 -0.0388 1.1817 7.97e-04 0.0542
6 -0.008768  0.00404 -0.0119 1.1879 7.55e-05 0.0565
7  0.046037 -0.02415  0.0586 1.1855 1.81e-03 0.0603
8  0.105490 -0.07298  0.1146 1.2038 6.89e-03 0.0841
9 -0.031520  0.00886 -0.0520 1.1751 1.43e-03 0.0515
10 -0.141287  0.07985 -0.1723 1.1381 1.53e-02 0.0637
11 -0.042751  0.00795 -0.0778 1.1651 3.18e-03 0.0505
12  0.049451 -0.00330  0.1009 1.1538 5.33e-03 0.0501
13 -0.065145  0.01211 -0.1185 1.1449 7.33e-03 0.0505
14  0.039126 -0.01656  0.0555 1.1789 1.62e-03 0.0549
15  0.113395 -0.24020 -0.3232 1.1473 5.27e-02 0.1117
16  0.000938 -0.07317 -0.1568 1.1485 1.28e-02 0.0639
17  0.291075 -0.48246 -0.5665 1.1644 1.57e-01 0.1820
18  0.005627 -0.11284 -0.2339 1.0958 2.77e-02 0.0652
19 -4.989462  7.43028  8.2028 0.0123 3.17e+00 0.2786  *
20  0.352440 -0.55157 -0.6264 1.2351 1.92e-01 0.2226

> summary(influence.measures(lm.WatershedI))
Potentially influential observations of
      lm(formula = salt_concentrationI ~ roadway_area, data = WatershedI) :

      dfb.1_ dfb.rdw_ dffit   cov.r   cook.d   hat
19 -4.99_*  7.43_*  8.20_*  0.01_*  3.17_*  0.28
> case(lm.WatershedI)

      e          h          si      sta.res      stu.res      dffit
1  1.0359337 0.15950451 5.897406  0.19694575  0.19160342  0.19659366
2  4.1372544 0.17375800 5.799645  0.79330640  0.78479675  0.87006111
3  1.8233867 0.06757603 5.885979  0.32912012  0.32081401  0.13214723
4  2.3789998 0.09897621 5.872394  0.43682616  0.42678692  0.26132982
5 -0.9309057 0.05418887 5.899204 -0.16683450 -0.16225949 -0.05333489
6 -0.2799152 0.05646093 5.903356 -0.05022603 -0.04881435 -0.01674999
7  1.3214056 0.06025314 5.894506  0.23758174  0.23125082  0.08472371
8  2.1266885 0.08413972 5.879117  0.38732153  0.37798730  0.19537804
9 -1.2825566 0.05149182 5.895124 -0.22952943 -0.22338967 -0.06962637
10 -3.7276039 0.06366940 5.829371 -0.67142500 -0.66083563 -0.25347277
11 -1.9335472 0.05052742 5.884121 -0.34585649 -0.33723448 -0.10289624
12  2.5154623 0.05005340 5.870492  0.44983223  0.43963645  0.13254159
13 -2.9335472 0.05052742 5.858442 -0.52472799 -0.51388948 -0.15611254
14  1.3194245 0.05489174 5.894586  0.23655172  0.23024511  0.07663196
15 -4.9507168 0.11172589 5.764670 -0.91553920 -0.91121348 -0.62269433
16 -3.3924622 0.06391459 5.842199 -0.61113849 -0.60017919 -0.23163252
17 -6.1579808 0.18196901 5.668128 -1.18668599 -1.20119588 -1.36982788
18 -4.9427923 0.06517321 5.772103 -0.89102349 -0.88567153 -0.34459608
19 19.7347551 0.27858326 1.760203  4.04968272 13.20006228  7.62079962
20 -5.8612827 0.22261543 5.679343 -1.15866282 -1.17051400 -1.67846391

      dffits      cook      (Intercept) roadway_area
1  0.08346839 3.680449e-03  0.0829190218 -0.069159454
2  0.35989512 6.617431e-02  0.3585417814 -0.303731751
3  0.08636603 3.925167e-03  0.0733492924 -0.044046060
4  0.14145177 1.048050e-02  0.1346609004 -0.099502862
5 -0.03883854 7.973470e-04 -0.0267903259  0.010798323
6 -0.01194102 7.547721e-05 -0.0087675025  0.004039383
7  0.05855550 1.809526e-03  0.0460373315 -0.024154959
8  0.11456795 6.891046e-03  0.1054899563 -0.072978107
9 -0.05204891 1.430026e-03 -0.0315198276  0.008859332
10 -0.17232339 1.532733e-02 -0.1412866500  0.079846080
11 -0.07779547 3.182780e-03 -0.0427507127  0.007948225
12  0.10091620 5.330961e-03  0.0494514744 -0.003296089
13 -0.11854741 7.326274e-03 -0.0651450034  0.012111779
14  0.05548852 1.624979e-03  0.0391262095 -0.016564609
15 -0.32316443 5.271458e-02  0.1133947487 -0.240204050
16 -0.15682785 1.275069e-02  0.0009378214 -0.073174266
17 -0.56653621 1.566280e-01  0.2910749437 -0.482464022
18 -0.23385231 2.767491e-02  0.0056269355 -0.112835523
19  8.20276709 3.166510e+00 -4.9894623899  7.430278350
20 -0.62637770 1.922222e-01  0.3524403558 -0.551567236

attr(,"class")
[1] "case" "matrix" "array"
> cooks.distance(lm.WatershedI)

      1          2          3          4          5          6
3.680449e-03 6.617431e-02 3.925167e-03 1.048050e-02 7.973470e-04 7.547721e-05
      7          8          9         10         11         12
1.809526e-03 6.891046e-03 1.430026e-03 1.532733e-02 3.182780e-03 5.330961e-03

```

```

      13      14      15      16      17      18
7.326274e-03 1.624979e-03 5.271458e-02 1.275069e-02 1.566280e-01 2.767491e-02
      19      20
3.166510e+00 1.922222e-01
> covratio(lm.WatershedI)
      1      2      3      4      5      6      7
1.32812228 1.26365283 1.18793033 1.21801851 1.18167648 1.18786077 1.18551827
      8      9     10     11     12     13     14
1.20378382 1.17506018 1.13811728 1.16512730 1.15379415 1.14492117 1.17885723
     15     16     17     18     19     20
1.14730852 1.14846914 1.16444283 1.09580869 0.01227987 1.23505389
> dfbetas(lm.WatershedI)
      (Intercept) roadway_area
1    0.0829190218 -0.069159454
2    0.3585417814 -0.303731751
3    0.0733492924 -0.044046060
4    0.1346609004 -0.099502862
5   -0.0267903259  0.010798323
6   -0.0087675025  0.004039383
7    0.0460373315 -0.024154959
8    0.1054899563 -0.072978107
9   -0.0315198276  0.008859332
10  -0.1412866500  0.079846080
11  -0.0427507127  0.007948225
12   0.0494514744 -0.003296089
13  -0.0651450034  0.012111779
14   0.0391262095 -0.016564609
15   0.1133947487 -0.240204050
16   0.0009378214 -0.073174266
17   0.2910749437 -0.482464022
18   0.0056269355 -0.112835523
19  -4.9894623899  7.430278350
20   0.3524403558 -0.551567236
> dffits(lm.WatershedI)
      1      2      3      4      5      6
0.08346839 0.35989512 0.08636603 0.14145177 -0.03883854 -0.01194102
      7      8      9     10     11     12
0.05855550 0.11456795 -0.05204891 -0.17232339 -0.07779547 0.10091620
     13     14     15     16     17     18
-0.11854741 0.05548852 -0.32316443 -0.15682785 -0.56653621 -0.23385231
     19     20
8.20276709 -0.62637770
> hatvalues(lm.WatershedI)
      1      2      3      4      5      6      7
0.15950451 0.17375800 0.06757603 0.09897621 0.05418887 0.05646093 0.06025314
      8      9     10     11     12     13     14
0.08413972 0.05149182 0.06366940 0.05052742 0.05005340 0.05052742 0.05489174
     15     16     17     18     19     20
0.11172589 0.06391459 0.18196901 0.06517321 0.27858326 0.22261543
> ols_plot_cooksd_chart(lm.WatershedI)
> ols_plot_dfbetas(lm.WatershedI)
> ols_plot_dffits(lm.WatershedI)
> ols_plot_diagnostics(lm.WatershedI)
> ols_plot_hadi(lm.WatershedI)
> ols_plot_resid_box(lm.WatershedI)
> ols_plot_resid_fit(lm.WatershedI)
> ols_plot_resid_fit_spread(lm.WatershedI)
> ols_plot_resid_hist(lm.WatershedI)
> ols_plot_resid_lev(lm.WatershedI)
> ols_plot_resid_pot(lm.WatershedI)
> ols_plot_resid_qq(lm.WatershedI)
> ols_plot_resid_regressor(lm.WatershedI)
> ols_plot_resid_spread(lm.WatershedI)
> ols_plot_resid_stand(lm.WatershedI)
> ols_plot_resid_stud(lm.WatershedI)
> ols_plot_resid_stud_fit(lm.WatershedI)
> # To analyze this data set, one possible way is first to remove (x19, y19)
> # and then to analyze the remaining data set in Table 6-1.
> # $ 6-2.6 Correlation and Regression
> # Refer to $ 6-2.1 and 6-2.5.
> # EXAMPLE 6-6 (continued)
> # 1. Run the correlation analysis
> cor_results <- cor.test(roadway_area, salt_concentration,
+ alternative = "two.sided",

```

```

+ method = "pearson",
+ conf.level = 0.95)
> # 2. Extract and print outputs with 4 significant figures
> cat("--- cor.test() Outputs ---\n")
--- cor.test() Outputs ---
> cat(sprintf("Sample Correlation Coefficient (r): %s\n",
+ signif(cor_results$estimate, 4)))
Sample Correlation Coefficient (r): 0.9754
> cat(sprintf("Calculated Test Statistic (t0): %s\n",
+ signif(cor_results$statistic, 4)))
Calculated Test Statistic (t0): 18.77
> cat(sprintf("Degrees of Freedom (df): %d\n", cor_results$parameter))
Degrees of Freedom (df): 18
> cat(sprintf("Exact P-Value: %.3e\n", cor_results$p.value))
Exact P-Value: 2.863e-13
> cat(sprintf("Approximate 95% CI Lower Limit: %s\n",
+ signif(cor_results$conf.int[1], 4)))
Approximate 95% CI Lower Limit: 0.9376
> cat(sprintf("Approximate 95% CI Upper Limit: %s\n",
+ signif(cor_results$conf.int[2], 4)))
Approximate 95% CI Upper Limit: 0.9904
> # 3. Derive the exact two-tailed critical value for comparison
> alpha <- 0.05
> df <- cor_results$parameter
> t_critical <- qt(1 - alpha/2, df)
> cat(sprintf("Two-Tailed Critical Value (t_{alpha/2}): %s\n",
+ signif(t_critical, 4)))
Two-Tailed Critical Value (t_{alpha/2}): 2.101
> # 4. Fit the ordinary least-squares simple linear regression model
> regression_model <- lm(salt_concentration ~ roadway_area)
> model_summary <- summary(regression_model)
> # 5. Extract regression slope coefficients and statistics
> slope_estimate <- model_summary$coefficients["roadway_area", "Estimate"]
> slope_se <- model_summary$coefficients["roadway_area", "Std. Error"]
> slope_t0 <- model_summary$coefficients["roadway_area", "t value"]
> slope_pvalue <- model_summary$coefficients["roadway_area", "Pr(>|t|)"]
> residual_df <- model_summary$df[2]
> # 6. Print out the structured comparison table
> cat("=====\n")
=====
> cat(" STATISTICAL METRIC COMPARISON SUMMARY \n")
STATISTICAL METRIC COMPARISON SUMMARY
> cat("=====\n")
=====
> cat(sprintf("Degrees of Freedom (DF): %d\n", residual_df))
Degrees of Freedom (DF): 18
> cat(sprintf("Slope Estimate (betal_hat): %s\n",
+ signif(slope_estimate, 4)))
Slope Estimate (betal_hat): 17.55
> cat(sprintf("Slope Estimated Standard Error (SE): %s\n",
+ signif(slope_se, 4)))
Slope Estimated Standard Error (SE): 0.9346
> cat("\n--- Test Statistic (t0) Verification ---\n")

--- Test Statistic (t0) Verification ---
> cat(sprintf("Regression Slope T-Test Statistic: %s\n",
+ signif(slope_t0, 4)))
Regression Slope T-Test Statistic: 18.77
> cat(sprintf("Correlation Coefficient Test Statistic: %s\n",
+ signif(cor_results$statistic, 4)))
Correlation Coefficient Test Statistic: 18.77
> cat("\n--- P-Value Verification ---\n")

--- P-Value Verification ---
> cat(sprintf("Regression Slope T-Test P-Value: %.3e\n", slope_pvalue))
Regression Slope T-Test P-Value: 2.863e-13
> cat(sprintf("Correlation Coefficient Test P-Value: %.3e\n",
+ cor_results$p.value))
Correlation Coefficient Test P-Value: 2.863e-13
> cat("=====\n")
=====
> spm(Watershed, T, T, T, T, T)
> sp(roadway_area, salt_concentration, "xy", T, T, T, T,
+ xlab = "Roadway area (%)", ylab = "Salt conc. (mg/l)")

```

```
[1] 10 19
> sp(salt_concentration, roadway_area, "xy", T, T, T, T,
+ xlab = "Salt conc. (mg/l)", ylab = "Roadway area (%)")
[1] 10 19
> bivariateNormal(cor(roadway_area, salt_concentration))
> ad.test(roadway_area); cvm.test(roadway_area)

Anderson-Darling normality test

data: roadway_area
A = 0.64036, p-value = 0.08115

Cramer-von Mises normality test

data: roadway_area
W = 0.11974, p-value = 0.05511

> lillie.test(roadway_area); pearson.test(roadway_area)

Lilliefors (Kolmogorov-Smirnov) normality test

data: roadway_area
D = 0.2127, p-value = 0.0182

Pearson chi-square normality test

data: roadway_area
P = 5.9, p-value = 0.2067

> sf.test(roadway_area); shapiro.test(roadway_area)

Shapiro-Francia normality test

data: roadway_area
W = 0.93244, p-value = 0.1515

Shapiro-Wilk normality test

data: roadway_area
W = 0.9277, p-value = 0.1394

> qqplot(roadway_area)
[1] 19 20
> ad.test(salt_concentration); cvm.test(salt_concentration)

Anderson-Darling normality test

data: salt_concentration
A = 0.55589, p-value = 0.1317

Cramer-von Mises normality test

data: salt_concentration
W = 0.10206, p-value = 0.09723

> lillie.test(salt_concentration); pearson.test(salt_concentration)

Lilliefors (Kolmogorov-Smirnov) normality test

data: salt_concentration
D = 0.17697, p-value = 0.1032

Pearson chi-square normality test

data: salt_concentration
P = 10.8, p-value = 0.02891

> sf.test(salt_concentration); shapiro.test(salt_concentration)
```

## Shapiro-Francia normality test

```
data: salt_concentration
W = 0.94404, p-value = 0.2443
```

## Shapiro-Wilk normality test

```
data: salt_concentration
W = 0.93948, p-value = 0.2344
```

```
> qqplot(salt_concentration)
[1] 20 19
> qqplot(lm(salt_concentration ~ roadway_area), reps = 10000)
[1] 10 12
> qqplot(lm(roadway_area ~ salt_concentration), reps = 10000)
[1] 10 12
> # 7. Input the baseline data
> r <- 0.9754034
> n <- 20
> alpha <- 0.05
> # 8. Step 1: Apply Fisher's Z-transformation to the sample r
> # This can be calculated via log((1+r)/(1-r)) * 0.5 or R's built-in atanh()
> # function
> Z_r <- atanh(r)
> # 9. Step 2: Calculate the standard error in the Z-scale
> se_Z <- 1 / sqrt(n - 3)
> # 10. Step 3: Construct the 95% confidence limits in the Z-scale
> z_critical <- qnorm(1 - alpha/2)
> Z_L <- Z_r - z_critical * se_Z
> Z_U <- Z_r + z_critical * se_Z
> # 11. Step 4: Back-transform the limits back to the rho scale
> # This can be done via (exp(2*Z) - 1) / (exp(2*Z) + 1) or R's built-in
> # tanh() function
> rho_L <- tanh(Z_L)
> rho_U <- tanh(Z_U)
> # Print high-precision numerical results
> cat(sprintf("Fisher's Z_r: %.3f\n", Z_r))
Fisher's Z_r: 2.193
> cat(sprintf("95%% Confidence Interval for rho: [%.4f, %.4f]\n", rho_L,
+ rho_U))
95% Confidence Interval for rho: [0.9376, 0.9904]
> # EXERCISES FOR § 6-2
> # § 6-3 Multiple Regression
> # Refer to § 1-3, 6-1 and 6-2.
> # --- 1. Setup ---
> if(!require(pacman)) install.packages("pacman")
> pacman::p_load(broom, car, ggfortify, ggplot2, goftest, HH, lmtest, mgcv,
+ nortest, olsrr, patchwork, rgl)
> # --- 2. Data Preparation ---
> # Wire bond data
> (WB_df <- data.frame(
+ PullStr = c( 9.95, 24.45, 31.75, 35.00, 25.02, 16.86, 14.38, 9.60,
+ 24.35, 27.50, 17.08, 37.00, 41.95, 11.66, 21.65, 17.89,
+ 69.00, 10.30, 34.93, 46.59, 44.88, 54.12, 56.63, 22.13,
+ 21.15),
+ WireLn = c( 2, 8, 11, 10, 8, 4, 2, 2, 9, 8, 4, 11, 12, 2, 4,
+ 4, 20, 1, 10, 15, 15, 16, 17, 6, 5),
+ DieHt = c( 50, 110, 120, 550, 295, 200, 375, 52, 100, 300, 412, 400,
+ 500, 360, 205, 400, 600, 585, 540, 250, 290, 510, 590, 100,
+ 400)
+ ))
  PullStr WireLn DieHt
1    9.95      2    50
2   24.45      8   110
3   31.75     11   120
4   35.00     10   550
5   25.02      8   295
6   16.86      4   200
7   14.38      2   375
8    9.60      2    52
9   24.35      9   100
10  27.50      8   300
11  17.08      4   412
```

```

12  37.00      11  400
13  41.95      12  500
14  11.66       2  360
15  21.65       4  205
16  17.89       4  400
17  69.00      20  600
18  10.30       1  585
19  34.93      10  540
20  46.59      15  250
21  44.88      15  290
22  54.12      16  510
23  56.63      17  590
24  22.13       6  100
25  21.15       5  400
> # --- 3. Exploratory Analysis (Fig 1-11 / 6-17) ---
> scatterplotMatrix(~ PullStr + WireLn + DieHt, data = WB_df,
+ smooth = list(lty.smooth = 1, col.smooth = "red"),
+ regLine = list(col = "blue"),
+ main = "Figure 1-11: Wire Bond Scatterplot Matrix")
> # --- 4. 3D Visualization (Fig 1-12) ---
> # Note: PullStr is the dependent variable (Y-axis in car::scatter3d)
> scatter3d(PullStr ~ WireLn + DieHt, data = WB_df,
+ surface = FALSE, grid = TRUE, sphere.size = 1,
+ xlab = "Wire Length", ylab = "Pull Strength",
+ zlab = "Die Height")
> # --- 5. Modeling & Selection ---
> lm_lin <- lm(PullStr ~ WireLn + DieHt, data = WB_df)
> summary(lm_lin)

Call:
lm(formula = PullStr ~ WireLn + DieHt, data = WB_df)

Residuals:
    Min       1Q   Median       3Q      Max
-3.865 -1.542 -0.362  1.196  5.841

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.263791   1.060066   2.136 0.044099 *
WireLn       2.744270   0.093524  29.343 < 2e-16 ***
DieHt        0.012528   0.002798   4.477 0.000188 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.288 on 22 degrees of freedom
Multiple R-squared:  0.9811,    Adjusted R-squared:  0.9794
F-statistic: 572.2 on 2 and 22 DF,  p-value: < 2.2e-16

> # Check for non-linearity
> residualPlots(lm_lin, smooth = TRUE, id = TRUE)
      Test stat Pr(>|Test stat|)
WireLn      2.8602      0.0093708 **
DieHt       0.6103      0.5481974
Tukey test   3.5167      0.0004369 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> # Automated Stepwise Selection (AIC based)
> # Using ols_step_both_aic requires a model object
> lm_quad_full <- lm(PullStr ~ (WireLn + DieHt)^2 + I(WireLn^2) + I(DieHt^2),
+ data = WB_df)
> summary(lm_quad_full)

Call:
lm(formula = PullStr ~ (WireLn + DieHt)^2 + I(WireLn^2) + I(DieHt^2),
    data = WB_df)

Residuals:
    Min       1Q   Median       3Q      Max
-2.5122 -1.1739 -0.0084  0.5814  4.6115

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  6.469e+00  1.444e+00   4.480 0.000256 ***
WireLn       1.656e+00  2.700e-01   6.135 6.75e-06 ***

```



```
DieHt      1.666e-02  8.701e-03  1.914 0.070745 .
I(WireLn^2) 2.660e-02  1.505e-02  1.768 0.093101 .
I(DieHt^2)  -2.643e-05  1.478e-05  -1.788 0.089790 .
WireLn:DieHt 1.482e-03  4.629e-04  3.201 0.004699 **
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.674 on 19 degrees of freedom
Multiple R-squared:  0.9913,    Adjusted R-squared:  0.989
F-statistic: 431.8 on 5 and 19 DF,  p-value: < 2.2e-16
```

```
> (step_aic_result <- ols_step_both_aic(lm_quad_full))
```

#### Stepwise Summary

| Step | Variable         | AIC     | SBC     | SBIC    | R2      | Adj. R2 |
|------|------------------|---------|---------|---------|---------|---------|
| 0    | Base Model       | 212.400 | 214.838 | 138.132 | 0.00000 | 0.00000 |
| 1    | WireLn (+)       | 131.326 | 134.983 | 59.187  | 0.96395 | 0.96239 |
| 2    | WireLn:DieHt (+) | 104.169 | 109.045 | -89.506 | 0.98877 | 0.98775 |

#### Final Model Output

##### Model Summary

|                |       |           |         |
|----------------|-------|-----------|---------|
| R              | 0.994 | RMSE      | 1.656   |
| R-Squared      | 0.989 | MSE       | 2.743   |
| Adj. R-Squared | 0.988 | Coef. Var | 6.081   |
| Pred R-Squared | 0.985 | AIC       | 104.169 |
| MAE            | 1.231 | SBC       | 109.045 |

RMSE: Root Mean Square Error

MSE: Mean Square Error

MAE: Mean Absolute Error

AIC: Akaike Information Criteria

SBC: Schwarz Bayesian Criteria

##### ANOVA

|            | Sum of Squares | DF | Mean Square | F       | Sig.   |
|------------|----------------|----|-------------|---------|--------|
| Regression | 6037.381       | 2  | 3018.691    | 968.607 | 0.0000 |
| Residual   | 68.564         | 22 | 3.117       |         |        |
| Total      | 6105.945       | 24 |             |         |        |

##### Parameter Estimates

| model        | Beta  | Std. Error | Std. Beta | t      | Sig.  | lower | upper |
|--------------|-------|------------|-----------|--------|-------|-------|-------|
| (Intercept)  | 6.950 | 0.705      |           | 9.860  | 0.000 | 5.488 | 8.412 |
| WireLn       | 2.091 | 0.134      | 0.707     | 15.590 | 0.000 | 1.813 | 2.370 |
| WireLn:DieHt | 0.002 | 0.000      | 0.316     | 6.973  | 0.000 | 0.001 | 0.002 |

```
> plot(step_aic_result)
> # Final model (Empirical model usually focuses on main effects +
> # interaction)
> lm_final <- lm(PullStr ~ WireLn + DieHt + I(WireLn*DieHt), data = WB_df)
> summary(lm_final)
```

```
Call:
lm(formula = PullStr ~ WireLn + DieHt + I(WireLn * DieHt), data = WB_df)
```

#### Residuals:

```
      Min       1Q   Median       3Q      Max
-2.8113 -1.1029 -0.0058  0.5374  5.1170
```

#### Coefficients:

```
      Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept)      6.6029761  1.4158295   4.664 0.000133 ***
WireLn           2.1243159  0.1791705  11.856 9.09e-11 ***
DieHt            0.0010646  0.0037394   0.285 0.778665
I(WireLn * DieHt) 0.0014811  0.0003901   3.796 0.001056 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.803 on 21 degrees of freedom
Multiple R-squared:  0.9888,    Adjusted R-squared:  0.9872
F-statistic: 618.8 on 3 and 21 DF,  p-value: < 2.2e-16
```

```
> # --- 6. Diagnostics ---
> vif(lm_lin) # Check multicollinearity on linear terms
WireLn      DieHt
1.167128 1.167128
> par(mfrow = c(2, 2), oma = c(0, 0.3, 1.5, 0))
> plot(lm_final)
> par(mfrow = c(1, 1))
> qqPlot(lm_final, main = "Residual Normality", reps = 10000)
[1] 9 15
> # --- 7. 3D Surface Visualizations (Figure 1-13) ---
> # This visualizes the regression plane/surface
> scatter3d(PullStr ~ WireLn + DieHt, data = WB_df,
+ fit = c("linear", "quadratic"),
+ surface.col = c("steelblue", "orange"), alpha = 0.5,
+ xlab = "Wire Length", ylab = "Pull Strength",
+ zlab = "Die Height", id = TRUE)
> # § 6-3.1 Estimation of Parameters in Multiple Regression
> # Refer to § 6-2.1.
> # Table 6-4 Data for Multiple Linear Regression
> # EXAMPLE 6-7 Wire Bond Pull Strength
> # Refer to Table 1-1 Wire Bond Data & Figures 1-11~13 in § 1-3.
> # Table 1-1 Wire Bond Data
> # Figure 1-11 Scatter plots of the wire bond pull strength data in Table
> # 1-1.
> # Figure 1-12 Three-dimensional scatter diagram of the wire and pull
> # strength data.
> # Figure 1-13 Plot of estimated values of pull strength from the empirical
> # model in equation 1-6.
> # Refer to Table 2-9 Wire Bond Data & Figure 2-19 in § 2-6.
> # Table 2-9 Wire Bond Data
> # Figure 2-19 Scatter diagrams and box plots for the wire bond pull
> # strength data in Table 1-1. (a) Pull strength versus length. (b) Pull
> # strength versus die height.
> # Table 6-5 Wire Bond Pull Strength Data for Example 6-7
> PullStr <- WB_df$PullStr
> WireLn <- WB_df$WireLn
> DieHt <- WB_df$DieHt
> summary(WB_df)
      PullStr      WireLn      DieHt
Min.   : 9.60   Min.   : 1.00   Min.   : 50.0
1st Qu.:17.08   1st Qu.: 4.00   1st Qu.:200.0
Median :24.45   Median : 8.00   Median :360.0
Mean   :29.03   Mean   : 8.24   Mean   :331.8
3rd Qu.:37.00   3rd Qu.:11.00   3rd Qu.:500.0
Max.   :69.00   Max.   :20.00   Max.   :600.0
> # Figure 6-17 Matrix of scatter plots (from Minitab) for the wire bond pull
> # strength data in Table 6-5.
> sp(WireLn, PullStr, "xy", xlab = "Wire length", ylab = "Pull strength")
> sp(DieHt, PullStr, "xy", xlab = "Die height", ylab = "Pull strength")
> scatter3d(WireLn, PullStr, DieHt, "Wire length", "Pull strength",
+ "Die height", T, T, surface = F)
> scatter3d(WireLn, PullStr, DieHt, "Wire length", "Pull strength",
+ "Die height", T, T, model.summary = T)
$linear
```

```
Call:
lm(formula = y ~ x + z)
```

```
Residuals:
      Min       1Q   Median       3Q      Max
-0.063990 -0.025522 -0.005993  0.019805  0.096704
```

```
Coefficients:
```

```

      Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.06565    0.01558  -4.214 0.000358 ***
x            0.86326    0.02942  29.343 < 2e-16 ***
z            0.11408    0.02548   4.477 0.000188 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.03788 on 22 degrees of freedom
Multiple R-squared:  0.9811,    Adjusted R-squared:  0.9794
F-statistic: 572.2 on 2 and 22 DF,  p-value: < 2.2e-16

> scatter3d(WireLn, PullStr, DieHt, "Wire length", "Pull strength",
+ "Die height", T, T, fit = "q", model.summary = T)
$quadratic

```

```

Call:
lm(formula = y ~ (x + z)^2 + I(x^2) + I(z^2))

```

```

Residuals:
      Min       1Q   Median       3Q      Max
-0.041592 -0.019435 -0.000139  0.009626  0.076349

```

```

Coefficients:
      Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.01006    0.01820  -0.552  0.5871
x            0.56103    0.07596   7.386 5.37e-07 ***
z            0.14111    0.06728   2.097  0.0496 *
I(x^2)       0.15900    0.08993   1.768  0.0931 .
I(z^2)      -0.13236    0.07404  -1.788  0.0898 .
x:z          0.25639    0.08009   3.201  0.0047 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

Residual standard error: 0.02772 on 19 degrees of freedom
Multiple R-squared:  0.9913,    Adjusted R-squared:  0.989
F-statistic: 431.8 on 5 and 19 DF,  p-value: < 2.2e-16

```

```

> ols_regress(PullStr ~ WireLn + DieHt + I(WireLn*DieHt), WB_df)
      Model Summary

```

|                |       |           |         |
|----------------|-------|-----------|---------|
| R              | 0.994 | RMSE      | 1.653   |
| R-Squared      | 0.989 | MSE       | 2.732   |
| Adj. R-Squared | 0.987 | Coef. Var | 6.212   |
| Pred R-Squared | 0.984 | AIC       | 106.073 |
| MAE            | 1.212 | SBC       | 112.167 |

```

RMSE: Root Mean Square Error
MSE: Mean Square Error
MAE: Mean Absolute Error
AIC: Akaike Information Criteria
SBC: Schwarz Bayesian Criteria

```

## ANOVA

|            | Sum of Squares | DF | Mean Square | F       | Sig.   |
|------------|----------------|----|-------------|---------|--------|
| Regression | 6037.645       | 3  | 2012.548    | 618.793 | 0.0000 |
| Residual   | 68.300         | 21 | 3.252       |         |        |
| Total      | 6105.945       | 24 |             |         |        |

## Parameter Estimates

| model             | Beta  | Std. Error | Std. Beta | t      | Sig   | lower  | upper |
|-------------------|-------|------------|-----------|--------|-------|--------|-------|
| (Intercept)       | 6.603 | 1.416      |           | 4.664  | 0.000 | 3.659  | 9.547 |
| WireLn            | 2.124 | 0.179      | 0.885     | 11.856 | 0.000 | 1.752  | 2.497 |
| DieHt             | 0.001 | 0.004      | 0.150     | 0.285  | 0.779 | -0.007 | 0.009 |
| I(WireLn * DieHt) | 0.001 | 0.000      | 0.097     | 3.796  | 0.001 | 0.001  | 0.002 |

```

> (lm.WB <- lm(PullStr ~ WireLn + DieHt + I(WireLn*DieHt), WB_df))

```

```
Call:
lm(formula = PullStr ~ WireLn + DieHt + I(WireLn * DieHt), data = WB_df)

Coefficients:
      (Intercept)          WireLn          DieHt  I(WireLn * DieHt)
        6.602976         2.124316         0.001065         0.001481

> residualPlots(lm.WB, smooth = T, id = T)
      Test stat Pr(>|Test stat|)
WireLn      1.3980      0.1774
DieHt     -1.4225      0.1703
I(WireLn * DieHt)  0.7727      0.4487
Tukey test    1.3386      0.1807
> qqplot(lm.WB, reps = 10000)
[1] 9 15
> opar <- par(mfrow = c(2, 2), oma = c(0, 0, 1.1, 0)); plot(lm.WB); par(opar)
> summary(lm.WB)
```

```
Call:
lm(formula = PullStr ~ WireLn + DieHt + I(WireLn * DieHt), data = WB_df)
```

```
Residuals:
      Min       1Q   Median       3Q      Max
-2.8113 -1.1029 -0.0058  0.5374  5.1170
```

```
Coefficients:
      Estimate Std. Error t value Pr(>|t|)
(Intercept)  6.6029761  1.4158295   4.664 0.000133 ***
WireLn       2.1243159  0.1791705  11.856 9.09e-11 ***
DieHt        0.0010646  0.0037394   0.285 0.778665
I(WireLn * DieHt) 0.0014811  0.0003901   3.796 0.001056 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.803 on 21 degrees of freedom
Multiple R-squared:  0.9888,    Adjusted R-squared:  0.9872
F-statistic: 618.8 on 3 and 21 DF,  p-value: < 2.2e-16
```

```
> anova(lm.WB)
Analysis of Variance Table
```

```
Response: PullStr
      Df Sum Sq Mean Sq F value    Pr(>F)
WireLn  1 5885.9   5885.9 1809.707 < 2.2e-16 ***
DieHt   1  104.9    104.9  32.259 1.228e-05 ***
I(WireLn * DieHt) 1   46.9     46.9  14.412 0.001056 **
Residuals 21   68.3      3.3
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.WB, "I(WireLn * DieHt)")
```

```
Linear hypothesis test:
I(WireLn * DieHt)
```

```
Model 1: restricted model
Model 2: PullStr ~ WireLn + DieHt + I(WireLn * DieHt)
```

```
      Res.Df    RSS Df Sum of Sq      F    Pr(>F)
1         22 115.17
2         21  68.30  1    46.874 14.412 0.001056 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.WB, c("WireLn", "DieHt", "I(WireLn * DieHt)"))
```

```
Linear hypothesis test:
WireLn = 0
DieHt = 0
I(WireLn * DieHt)
```

```
Model 1: restricted model
Model 2: PullStr ~ WireLn + DieHt + I(WireLn * DieHt)
```

```
      Res.Df    RSS Df Sum of Sq      F    Pr(>F)
```

```

1      24 6105.9
2      21   68.3  3      6037.6 618.79 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> fitted(lm.WB)
      1      2      3      4      5      6      7      8
11.05295 25.01798 32.05325 36.57771 27.40695 16.49804 12.36165 11.06100
      9     10     11     12     13     14     15     16
27.16127 27.47152 17.97970 36.91312 41.51366 12.30125 16.53298 17.89583
      17     18     19     20     21     22     23     24
67.50124 10.21652 36.41895 44.28799 45.21923 53.22074 58.19988 20.34399
      25
20.61259
> predict(lm.WB, WB_df, T)
$fit
      1      2      3      4      5      6      7      8
11.05295 25.01798 32.05325 36.57771 27.40695 16.49804 12.36165 11.06100
      9     10     11     12     13     14     15     16
27.16127 27.47152 17.97970 36.91312 41.51366 12.30125 16.53298 17.89583
      17     18     19     20     21     22     23     24
67.50124 10.21652 36.41895 44.28799 45.21923 53.22074 58.19988 20.34399
      25
20.61259

$se.fit
      1      2      3      4      5      6      7      8
1.0038821 0.6215292 0.8439674 0.5756802 0.3920605 0.5316930 0.6201001 0.9987801
      9     10     11     12     13     14     15     16
0.7023335 0.3899449 0.5597927 0.4380210 0.5167107 0.6060940 0.5255304 0.5429978
      17     18     19     20     21     22     23     24
1.1837650 1.1244974 0.5594559 0.9149085 0.8105418 0.6869131 0.9173669 0.6032250
      25
0.4972079

$df
[1] 21

$residual.scale
[1] 1.803435

> resid(lm.WB)
      1      2      3      4      5      6
-1.102947218 -0.567975703 -0.303253224 -1.577706730 -2.386951905  0.361963185
      7      8      9     10     11     12
 2.018347475 -1.461000790 -2.811267747  0.028481171 -0.899701705  0.086875108
      13     14     15     16     17     18
 0.436340808 -0.641250741  5.117018258 -0.005833881  1.498755679  0.083481846
      19     20     21     22     23     24
-1.488950884  2.302014284 -0.339229079  0.899255579 -1.569883975  1.786009932
      25
 0.537410260
> confint(lm.WB)
              2.5 %      97.5 %
(Intercept)  3.658597527 9.547354736
WireLn       1.751710470 2.496921339
DieHt       -0.006711863 0.008841035
I(WireLn * DieHt) 0.000669758 0.002292442
> (newdata <- data.frame(WireLn = c(11, 5), DieHt = c(350, 200)))
  WireLn DieHt
1     11   350
2      5   200
> predict(lm.WB, newdata, T, NULL, Inf, "c")
$fit
      fit      lwr      upr
1 36.04529 35.09175 36.99883
2 18.91857 17.91739 19.91975

$se.fit
      1      2
0.4585178 0.4814262

$df
[1] 21

```

```

$residual.scale
[1] 1.803435

> predict(lm.WB, newdata, T, NULL, Inf, "p")
$fit
      fit      lwr      upr
1 36.04529 32.17552 39.91506
2 18.91857 15.03679 22.80035

$se.fit
      1      2
0.4585178 0.4814262

$df
[1] 21

$residual.scale
[1] 1.803435

> # EXAMPLE 6-7 (continued) Compute Estimate of Error Variance  $\sigma^2_{\hat{a}}$ 
> # Table 6-6 Minitab Regression Analysis Output for Wire Bond Pull Strength
> # Data
> # EXAMPLE 6-7 (continued)
> # § 6-3.2 Inferences in Multiple Regression
> # Refer to § 6-3.1.
> # EXAMPLE 6-7 (continued)
> # Table 6-7 Minitab Output
> # EXAMPLE 6-7 (continued)
> # § 6-3.3 Checking Model Adequacy
> # Refer to § 6-2.5 and 6-3.1.
> ols_test_breusch_pagan(lm.WB)

Breusch Pagan Test for Heteroskedasticity
-----
Ho: the variance is constant
Ha: the variance is not constant

      Data
-----
Response : PullStr
Variables: fitted values of PullStr

      Test Summary
-----
DF          =      1
Chi2         =    0.5875264
Prob > Chi2  =    0.4433773
> ols_test_f(lm.WB)

F Test for Heteroskedasticity
-----
Ho: Variance is homogenous
Ha: Variance is not homogenous

Variables: fitted values of PullStr

      Test Summary
-----
Num DF      =      1
Den DF      =     23
F           =    0.301067
Prob > F    =    0.5885016
> ols_test_normality(lm.WB)
-----
      Test          Statistic      pvalue
-----
Shapiro-Wilk      0.9379      0.1326
Kolmogorov-Smirnov 0.135      0.7025
Cramer-von Mises  2.2453      0.0000
Anderson-Darling  0.4295      0.2854
-----
> ols_test_outlier(lm.WB)
      studentized_residual unadjusted_p_val bonferroni_p_val
15          3.797357      0.001129469      0.02823672

```

```
> outlierTest(lm.WB)
      rstudent unadjusted p-value Bonferroni p
15 3.797357      0.0011295      0.028237
> ols_test_score(lm.WB)
```

# Score Test for Heteroskedasticity

```
-----
Ho: Variance is homogenous
Ha: Variance is not homogenous
```

Variables: fitted values of PullStr

## Test Summary

```
-----
DF          =      1
Chi2        =      0.3230185
Prob > Chi2 =      0.5697993
```

```
> influence.measures(lm.WB)
```

Influence measures of

```
lm(formula = PullStr ~ WireLn + DieHt + I(WireLn * DieHt), data = WB_df) :
```

|    | dfb.1     | dfb.WrLn  | dfb.DiHt  | dfb.I..D  | dfffit   | cov.r | cook.d   | hat    | inf |
|----|-----------|-----------|-----------|-----------|----------|-------|----------|--------|-----|
| 1  | -4.80e-01 | 0.356871  | 0.404584  | -0.342242 | -0.48773 | 1.586 | 6.08e-02 | 0.3099 | *   |
| 2  | -2.84e-02 | -0.040267 | 0.031341  | 0.029526  | -0.12052 | 1.350 | 3.79e-03 | 0.1188 |     |
| 3  | 2.27e-02  | -0.071457 | -0.009725 | 0.058387  | -0.09841 | 1.546 | 2.54e-03 | 0.2190 |     |
| 4  | 6.09e-02  | 0.009141  | -0.139403 | 0.002704  | -0.30980 | 1.147 | 2.42e-02 | 0.1019 |     |
| 5  | -1.34e-02 | -0.099945 | -0.050290 | 0.105041  | -0.30855 | 0.884 | 2.28e-02 | 0.0473 |     |
| 6  | 5.09e-02  | -0.031411 | -0.031142 | 0.023575  | 0.06331  | 1.320 | 1.05e-03 | 0.0869 |     |
| 7  | 1.04e-01  | -0.130043 | 0.126454  | -0.016602 | 0.44108  | 1.042 | 4.76e-02 | 0.1182 |     |
| 8  | -6.36e-01 | 0.473109  | 0.534829  | -0.452924 | -0.64629 | 1.458 | 1.05e-01 | 0.3067 |     |
| 9  | -3.85e-02 | -0.380516 | 0.096196  | 0.296995  | -0.75150 | 0.797 | 1.28e-01 | 0.1517 |     |
| 10 | 1.21e-04  | 0.001119  | 0.000628  | -0.001192 | 0.00350  | 1.275 | 3.21e-06 | 0.0468 |     |
| 11 | -3.68e-03 | 0.016444  | -0.079409 | 0.033478  | -0.16834 | 1.276 | 7.34e-03 | 0.0964 |     |
| 12 | -3.48e-03 | 0.005320  | 0.004435  | -0.004005 | 0.01213  | 1.291 | 3.86e-05 | 0.0590 |     |
| 13 | -1.32e-02 | 0.006160  | 0.020768  | 0.002059  | 0.07381  | 1.308 | 1.43e-03 | 0.0821 |     |
| 14 | -4.04e-02 | 0.045334  | -0.028543 | -0.002231 | -0.13192 | 1.333 | 4.54e-03 | 0.1129 |     |
| 15 | 9.20e-01  | -0.569798 | -0.548392 | 0.419732  | 1.15677  | 0.151 | 2.04e-01 | 0.0849 | *   |
| 16 | -6.88e-05 | 0.000128  | -0.000451 | 0.000181  | -0.00105 | 1.337 | 2.87e-07 | 0.0907 |     |
| 17 | 2.30e-01  | -0.338784 | -0.364362 | 0.607971  | 0.96361  | 1.683 | 2.30e-01 | 0.4309 | *   |
| 18 | -1.77e-02 | 0.008300  | 0.036872  | -0.022892 | 0.04609  | 1.987 | 5.58e-04 | 0.3888 | *   |
| 19 | 5.66e-02  | 0.002777  | -0.127454 | 0.006970  | -0.28167 | 1.162 | 2.01e-02 | 0.0962 |     |
| 20 | -4.36e-01 | 0.765195  | 0.261076  | -0.588176 | 0.89924  | 1.053 | 1.90e-01 | 0.2574 |     |
| 21 | 4.90e-02  | -0.084469 | -0.028880 | 0.061944  | -0.10350 | 1.510 | 2.81e-03 | 0.2020 |     |
| 22 | -1.30e-02 | 0.005026  | -0.011459 | 0.055209  | 0.21832  | 1.345 | 1.23e-02 | 0.1451 |     |
| 23 | -9.22e-02 | 0.180654  | 0.137225  | -0.324733 | -0.59771 | 1.343 | 8.92e-02 | 0.2588 |     |
| 24 | 2.41e-01  | -0.046626 | -0.211792 | 0.059082  | 0.37396  | 1.103 | 3.48e-02 | 0.1119 |     |
| 25 | 2.22e-03  | -0.005175 | 0.039131  | -0.017678 | 0.08697  | 1.292 | 1.98e-03 | 0.0760 |     |

```
> summary(influence.measures(lm.WB))
```

Potentially influential observations of

```
lm(formula = PullStr ~ WireLn + DieHt + I(WireLn * DieHt), data = WB_df) :
```

|    | dfb.1 | dfb.WrLn | dfb.DiHt | dfb.I(*D | dfffit | cov.r | cook.d | hat  |
|----|-------|----------|----------|----------|--------|-------|--------|------|
| 1  | -0.48 | 0.36     | 0.40     | -0.34    | -0.49  | 1.59  | 0.06   | 0.31 |
| 15 | 0.92  | -0.57    | -0.55    | 0.42     | 1.16   | 0.15  | 0.20   | 0.08 |
| 17 | 0.23  | -0.34    | -0.36    | 0.61     | 0.96   | 1.68  | 0.23   | 0.43 |
| 18 | -0.02 | 0.01     | 0.04     | -0.02    | 0.05   | 1.99  | 0.00   | 0.39 |

```
> case(lm.WB)
```

|    | e            | h          | si       | sta.res      | stu.res      | dfffit        |
|----|--------------|------------|----------|--------------|--------------|---------------|
| 1  | -1.102947218 | 0.30985906 | 1.823969 | -0.736182070 | -0.727894210 | -0.4952005700 |
| 2  | -0.567975703 | 0.11877412 | 1.843012 | -0.335494604 | -0.328290182 | -0.0765533758 |
| 3  | -0.303253224 | 0.21900307 | 1.846378 | -0.190274314 | -0.185848989 | -0.0850366802 |
| 4  | -1.577706730 | 0.10189700 | 1.810088 | -0.923129450 | -0.919736545 | -0.1790034986 |
| 5  | -2.386951905 | 0.04726123 | 1.765217 | -1.355988952 | -1.385347521 | -0.1184063078 |
| 6  | 0.361963185  | 0.08692020 | 1.846029 | 0.210043592  | 0.205197216  | 0.0344569158  |
| 7  | 2.018347475  | 0.11822856 | 1.784377 | 1.191838262  | 1.204567895  | 0.2706215073  |
| 8  | -1.461000790 | 0.30671749 | 1.805839 | -0.972959425 | -0.971664331 | -0.6463663494 |
| 9  | -2.811267747 | 0.15166508 | 1.717321 | -1.692458153 | -1.777326219 | -0.5025976487 |
| 10 | 0.028481171  | 0.04675254 | 1.847960 | 0.016175377  | 0.015785650  | 0.0013968746  |
| 11 | -0.899701705 | 0.09635033 | 1.835813 | -0.524804921 | -0.515549096 | -0.0959293833 |
| 12 | 0.086875108  | 0.05899140 | 1.847863 | 0.049658999  | 0.048465067  | 0.0054461609  |
| 13 | 0.436340808  | 0.08209065 | 1.845163 | 0.252537185  | 0.246826136  | 0.0390229148  |
| 14 | -0.641250741 | 0.11294806 | 1.841690 | -0.377531122 | -0.369689350 | -0.0816502694 |
| 15 | 5.117018258  | 0.08491698 | 1.408657 | 2.966102256  | 3.797356594  | 0.4748440332  |
| 16 | -0.005833881 | 0.09065565 | 1.847971 | -0.003392289 | -0.003310536 | -0.0005815996 |

```
17  1.498755679 0.43085365 1.793784  1.101585640  1.107512597  1.1345840189
18  0.083481846 0.38879054 1.847817  0.059210184  0.057788047  0.0531028299
19 -1.488950884 0.09623445 1.814482 -0.868464126 -0.863176744 -0.1585459580
20  2.302014284 0.25736774 1.748773  1.481223218  1.527522266  0.7977894959
21 -0.339229079 0.20199920 1.846020 -0.210567243 -0.205709867 -0.0858695901
22  0.899255579 0.14507827 1.835130  0.539286271  0.529972106  0.1526016209
23 -1.569883975 0.25875274 1.802431 -1.011079742 -1.011643441 -0.5480111826
24  1.786009932 0.11188129 1.798726  1.050867016  1.053618485  0.2249936689
25  0.537410260 0.07601071 1.843738  0.310007276  0.303230772  0.0442093161
      dffits      cook      (Intercept)      WireLn      DieHt
1  -0.487732244 6.083267e-02 -4.801422e-01  0.3568711657  0.4045838592
2  -0.120524458 3.792675e-03 -2.840792e-02 -0.0402670113  0.0313413211
3  -0.098414851 2.538056e-03  2.265094e-02 -0.0714573901 -0.0097248040
4  -0.309799751 2.417133e-02  6.086641e-02  0.0091411210 -0.1394026490
5  -0.308549124 2.280255e-02 -1.338546e-02 -0.0999453903 -0.0502895193
6   0.063310751 1.049955e-03  5.092844e-02 -0.0314111021 -0.0311419049
7   0.441077059 4.761470e-02  1.042834e-01 -0.1300431422  0.1264540373
8  -0.646294417 1.047027e-01 -6.361353e-01  0.4731089824  0.5348290756
9  -0.751495300 1.280248e-01 -3.846501e-02 -0.3805157642  0.0961962688
10  0.003495926 3.208104e-06  1.211077e-04  0.0011190031  0.0006275271
11 -0.168343590 7.341570e-03 -3.678697e-03  0.0164438059 -0.0794088781
12  0.012134625 3.864836e-05 -3.481214e-03  0.0053196225  0.0044351806
13  0.073813888 1.425885e-03 -1.324661e-02  0.0061600481  0.0207684268
14 -0.131917317 4.537068e-03 -4.036459e-02  0.0453342622 -0.0285427106
15  1.156773660 2.041015e-01  9.198750e-01 -0.5697977399 -0.5483918157
16 -0.001045277 2.868086e-07 -6.881925e-05  0.0001280465 -0.0004510452
17  0.963610636 2.296584e-01  2.301471e-01 -0.3387841296 -0.3643617124
18  0.046089376 5.575174e-04 -1.772404e-02  0.0083004341  0.0368721633
19 -0.281667755 2.007791e-02  5.659166e-02  0.0027765035 -0.1274542305
20  0.899244312 1.900909e-01 -4.360181e-01  0.7651953005  0.2610763547
21 -0.103497124 2.805873e-03  4.895867e-02 -0.0844686987 -0.0288799851
22  0.218318740 1.233828e-02 -1.296156e-02  0.0050261977 -0.0114585539
23 -0.597707083 8.921393e-02 -9.224684e-02  0.1806543712  0.1372249635
24  0.373961244 3.477939e-02  2.409003e-01 -0.0466261842 -0.2117917186
25  0.086971547 1.976476e-03  2.219972e-03 -0.0051746824  0.0391306279
I(WireLn * DieHt)
1  -0.3422423091
2   0.0295258552
3   0.0583872085
4   0.0027043056
5   0.1050412392
6   0.0235745125
7  -0.0166019774
8  -0.4529241899
9   0.2969953965
10 -0.0011920436
11  0.0334782722
12 -0.0040045015
13  0.0020593205
14 -0.0022306770
15  0.4197320975
16  0.0001805994
17  0.6079714120
18 -0.0228917559
19  0.0069698463
20 -0.5881763886
21  0.0619443998
22  0.0552091625
23 -0.3247328422
24  0.0590815975
25 -0.0176782362
attr(,"class")
[1] "case"      "matrix" "array"
> cooks.distance(lm.WB)
      1      2      3      4      5      6
6.083267e-02 3.792675e-03 2.538056e-03 2.417133e-02 2.280255e-02 1.049955e-03
      7      8      9     10     11     12
4.761470e-02 1.047027e-01 1.280248e-01 3.208104e-06 7.341570e-03 3.864836e-05
      13     14     15     16     17     18
1.425885e-03 4.537068e-03 2.041015e-01 2.868086e-07 2.296584e-01 5.575174e-04
      19     20     21     22     23     24
2.007791e-02 1.900909e-01 2.805873e-03 1.233828e-02 8.921393e-02 3.477939e-02
      25
1.976476e-03
```



```

> covratio(lm.WB)
      1      2      3      4      5      6      7      8
1.5863462 1.3500005 1.5456472 1.1467458 0.8843120 1.3200641 1.0416751 1.4578656
      9     10     11     12     13     14     15     16
0.7969645 1.2750579 1.2759183 1.2910992 1.3081987 1.3334523 0.1514180 1.3366813
      17     18     19     20     21     22     23     24
1.6831886 1.9873625 1.1618804 1.0526643 1.5103659 1.3446352 1.3430755 1.1026661
      25
1.2915820
> dfbetas(lm.WB)
      (Intercept)      WireLn      DieHt I(WireLn * DieHt)
1 -4.801422e-01  0.3568711657  0.4045838592 -0.3422423091
2 -2.840792e-02 -0.0402670113  0.0313413211  0.0295258552
3 -2.265094e-02 -0.0714573901 -0.0097248040  0.0583872085
4  6.086641e-02  0.0091411210 -0.1394026490  0.0027043056
5 -1.338546e-02 -0.0999453903 -0.0502895193  0.1050412392
6  5.092844e-02 -0.0314111021 -0.0311419049  0.0235745125
7  1.042834e-01 -0.1300431422  0.1264540373 -0.0166019774
8 -6.361353e-01  0.4731089824  0.5348290756 -0.4529241899
9 -3.846501e-02 -0.3805157642  0.0961962688  0.2969953965
10 1.211077e-04  0.0011190031  0.0006275271 -0.0011920436
11 -3.678697e-03  0.0164438059 -0.0794088781  0.0334782722
12 -3.481214e-03  0.0053196225  0.0044351806 -0.0040045015
13 -1.324661e-02  0.0061600481  0.0207684268  0.0020593205
14 -4.036459e-02  0.0453342622 -0.0285427106 -0.0022306770
15  9.198750e-01 -0.5697977399 -0.5483918157  0.4197320975
16 -6.881925e-05  0.0001280465 -0.0004510452  0.0001805994
17  2.301471e-01 -0.3387841296 -0.3643617124  0.6079714120
18 -1.772404e-02  0.0083004341  0.0368721633 -0.0228917559
19  5.659166e-02  0.0027765035 -0.1274542305  0.0069698463
20 -4.360181e-01  0.7651953005  0.2610763547 -0.5881763886
21  4.895867e-02 -0.0844686987 -0.0288799851  0.0619443998
22 -1.296156e-02  0.0050261977 -0.0114585539  0.0552091625
23 -9.224684e-02  0.1806543712  0.1372249635 -0.3247328422
24  2.409003e-01 -0.0466261842 -0.2117917186  0.0590815975
25  2.219972e-03 -0.0051746824  0.0391306279 -0.0176782362
> dffits(lm.WB)
      1      2      3      4      5      6
-0.487732244 -0.120524458 -0.098414851 -0.309799751 -0.308549124  0.063310751
      7      8      9     10     11     12
 0.441077059 -0.646294417 -0.751495300  0.003495926 -0.168343590  0.012134625
      13     14     15     16     17     18
 0.073813888 -0.131917317  1.156773660 -0.001045277  0.963610636  0.046089376
      19     20     21     22     23     24
-0.281667755  0.899244312 -0.103497124  0.218318740 -0.597707083  0.373961244
      25
 0.086971547
> hatvalues(lm.WB)
      1      2      3      4      5      6      7
0.30985906 0.11877412 0.21900307 0.10189700 0.04726123 0.08692020 0.11822856
      8      9     10     11     12     13     14
0.30671749 0.15166508 0.04675254 0.09635033 0.05899140 0.08209065 0.11294806
      15     16     17     18     19     20     21
0.08491698 0.09065565 0.43085365 0.38879054 0.09623445 0.25736774 0.20199920
      22     23     24     25
0.14507827 0.25875274 0.11188129 0.07601071
> ols_coll_diag(lm.WB)
Tolerance and Variance Inflation Factor
-----
      Variables  Tolerance  VIF
1      WireLn  0.14503223  6.895019
2      DieHt   0.29811306  3.354432
3 I(WireLn * DieHt) 0.08625121 11.594040

Eigenvalue and Condition Index
-----
      Eigenvalue Condition Index  intercept  WireLn  DieHt
1  3.52160777      1.000000  0.004349161  0.003148075  0.004817907
2  0.31995139      3.317634  0.097533928  0.008295124  0.009353386
3  0.14342688      4.955132  0.038101681  0.106703047  0.242159467
4  0.01501395     15.315210  0.860015230  0.881853753  0.743669241
      I(WireLn * DieHt)
1      0.003048711

```

```

2      0.056472787
3      0.008116372
4      0.932362130
> ols_plot_added_variable(lm.WB)
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
> ols_plot_cooksd_chart(lm.WB)
> ols_plot_dfbetas(lm.WB)
> ols_plot_dffits(lm.WB)
> ols_plot_diagnostics(lm.WB)
> ols_plot_hadi(lm.WB)
> ols_plot_resid_box(lm.WB)
> ols_plot_resid_fit(lm.WB)
> ols_plot_resid_fit_spread(lm.WB)
> ols_plot_resid_hist(lm.WB)
> ols_plot_resid_lev(lm.WB)
> ols_plot_resid_pot(lm.WB)
> ols_plot_resid_qq(lm.WB)
> ols_plot_resid_regressor(lm.WB)
> ols_plot_resid_spread(lm.WB)
> ols_plot_resid_stand(lm.WB)
> ols_plot_resid_stud(lm.WB)
> ols_plot_resid_stud_fit(lm.WB)
> # EXAMPLE 6-7 (continued)
> # Figure 6-18 Normal probability plot of residuals for wire bond empirical
> # model.
> # Figure 6-19 Plot of residuals against for wire bond empirical model.
> # Figure 6-20 Plot of residuals against x1 (wire length) for wire bond
> # empirical model.
> # Figure 6-21 Plot of residuals against x2 (die height) for wire bond
> # empirical model.
> # EXAMPLE 6-7 (continued)
> # Figure 6-22 A point that is remote in x-space.
> # EXAMPLE 6-8 Wire Bond Pull Strength
> # Table 6-8 Influence Diagnostics for the Wire Bond Pull Strength Data
> # EXAMPLE 6-8 (continued)
> # EXERCISES FOR § 6-3
> # § 6-4 OTHER ASPECTS OF REGRESSION
> # Refer to § 6-1, 6-2, and 6-3.
> # --- 1. Setup ---
> if(!require(pacman)) install.packages("pacman")
> pacman::p_load(broom, car, ggfortify, ggplot2, goftest, HH, lmtest, mgcv,
+ nortest, olsrr, patchwork, rgl)
> # § 6-4.1 Polynomial Models
> # EXAMPLE 6-9 Acetylene Yield
> # Table 6-9 The Acetylene Data
> Yld <- c(49.0, 50.2, 50.5, 48.5, 47.5, 44.5, 28.0, 31.5, 34.5, 35.0, 38.0,
+ 38.5, 15.0, 17.0, 20.5, 29.5)
> Temp <- c(1300, 1300, 1300, 1300, 1300, 1300, 1200, 1200, 1200, 1200, 1200,
+ 1200, 1100, 1100, 1100, 1100)
> Rto <- c(7.5, 9.0, 11.0, 13.5, 17.0, 23.0, 5.3, 7.5, 11.0, 13.5, 17.0,
+ 23.0, 5.3, 7.5, 11.0, 17.0)
> (C2H2 <- data.frame(Yld, Temp, Rto))
  Yld Temp  Rto
1  49.0 1300  7.5
2  50.2 1300  9.0
3  50.5 1300 11.0
4  48.5 1300 13.5
5  47.5 1300 17.0
6  44.5 1300 23.0
7  28.0 1200  5.3
8  31.5 1200  7.5
9  34.5 1200 11.0
10 35.0 1200 13.5
11 38.0 1200 17.0
12 38.5 1200 23.0
13 15.0 1100  5.3
14 17.0 1100  7.5
15 20.5 1100 11.0
16 29.5 1100 17.0
> summary(C2H2); var(C2H2); cor(C2H2)
      Yld      Temp      Rto
Min.   :15.00  Min.   :1100  Min.   : 5.30

```

```

1st Qu.:29.12    1st Qu.:1175    1st Qu.: 7.50
Median :36.50    Median :1200    Median :11.00
Mean   :36.11    Mean   :1212    Mean   :12.44
3rd Qu.:47.75    3rd Qu.:1300    3rd Qu.:17.00
Max.   :50.50    Max.   :1300    Max.   :23.00
      Yld      Temp      Rto
Yld  141.58062  906.5833  24.92971
Temp 906.58333 6500.0000 102.08333
Rto   24.92971 102.0833  32.05862
      Yld      Temp      Rto
Yld  1.0000000 0.9450377 0.3700350
Temp 0.9450377 1.0000000 0.2236278
Rto  0.3700350 0.2236278 1.0000000
> cor(Yld, Temp); cor(Yld, Rto); cor(Temp, Rto)
[1] 0.9450377
[1] 0.370035
[1] 0.2236278
> spm(C2H2)
> sp(Temp, Yld, "xy", xlab = "Temperature", ylab = "Yield")
> sp(Rto, Yld, "xy", xlab = "Ratio", ylab = "Yield")
> scatter3d(Temp, Yld, Rto, "Temperature", "Yield", "Ratio", T, T,
+ surface = F)
> scatter3d(Temp, Yld, Rto, "Temperature", "Yield", "Ratio", T, T,
+ model.summary = T)
$linear

```

```

Call:
lm(formula = y ~ x + z)

```

```

Residuals:
      Min       1Q   Median       3Q      Max
-0.198133 -0.056916  0.007026  0.062070  0.193375

```

```

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.28095     0.07657  -3.669  0.00283 **
x             1.13206     0.10063  11.250 4.51e-08 ***
z             0.17800     0.08597   2.071  0.05888 .
---

```

```

Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

Residual standard error: 0.1021 on 13 degrees of freedom
Multiple R-squared:  0.9196,    Adjusted R-squared:  0.9072
F-statistic: 74.35 on 2 and 13 DF,  p-value: 7.654e-08

```

```

> scatter3d(Temp, Yld, Rto, "Temperature", "Yield", "Ratio", T, T,
+ fit = "q", model.summary = T)
$squadratic

```

```

Call:
lm(formula = y ~ (x + z)^2 + I(x^2) + I(z^2))

```

```

Residuals:
      Min       1Q   Median       3Q      Max
-0.044415 -0.012635  0.000774  0.016487  0.042868

```

```

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.41501     0.06192  -6.702 5.35e-05 ***
x             0.92400     0.19581   4.719 0.000818 ***
z             1.21240     0.09786  12.389 2.16e-07 ***
I(x^2)       0.45178     0.14841   3.044 0.012375 *
I(z^2)      -0.21607     0.09299  -2.324 0.042508 *
x:z         -1.11569     0.12158  -9.177 3.47e-06 ***
---

```

```

Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

Residual standard error: 0.03004 on 10 degrees of freedom
Multiple R-squared:  0.9946,    Adjusted R-squared:  0.992
F-statistic: 371.5 on 5 and 10 DF,  p-value: 5.13e-11

```

```

> TempC <- Temp - mean(Temp); RtoC <- Rto - mean(Rto)

```

```
> C2H2C <- data.frame(Yld, TempC, RtoC)
> ols_regress(Yld ~ TempC + RtoC + I(TempC*RtoC), C2H2C)
```

## Model Summary

|                |       |           |        |
|----------------|-------|-----------|--------|
| R              | 0.992 | RMSE      | 1.421  |
| R-Squared      | 0.985 | MSE       | 2.019  |
| Adj. R-Squared | 0.981 | Coef. Var | 4.544  |
| Pred R-Squared | 0.971 | AIC       | 66.650 |
| MAE            | 1.112 | SBC       | 70.513 |

RMSE: Root Mean Square Error  
MSE: Mean Square Error  
MAE: Mean Absolute Error  
AIC: Akaike Information Criteria  
SBC: Schwarz Bayesian Criteria

## ANOVA

|            | Sum of Squares | DF | Mean Square | F       | Sig.   |
|------------|----------------|----|-------------|---------|--------|
| Regression | 2091.401       | 3  | 697.134     | 258.933 | 0.0000 |
| Residual   | 32.308         | 12 | 2.692       |         |        |
| Total      | 2123.709       | 15 |             |         |        |

## Parameter Estimates

| model           | Beta   | Std. Error | Std. Beta | t      | Sig   | lower  | upper  |
|-----------------|--------|------------|-----------|--------|-------|--------|--------|
| (Intercept)     | 36.835 | 0.423      |           | 87.163 | 0.000 | 35.914 | 37.755 |
| TempC           | 0.128  | 0.005      | 0.869     | 23.547 | 0.000 | 0.116  | 0.140  |
| RtoC            | 0.390  | 0.077      | 0.186     | 5.068  | 0.000 | 0.222  | 0.558  |
| I(TempC * RtoC) | -0.008 | 0.001      | -0.258    | -7.170 | 0.000 | -0.010 | -0.005 |

```
> (lm.C2H2C <- lm(Yld ~ TempC + RtoC + I(TempC*RtoC), C2H2C))
```

Call:

```
lm(formula = Yld ~ TempC + RtoC + I(TempC * RtoC), data = C2H2C)
```

Coefficients:

```
(Intercept)      TempC      RtoC  I(TempC * RtoC)
  36.83453      0.12830      0.39007      -0.00761
```

```
> residualPlots(lm.C2H2C, smooth = T, id = T)
```

Test stat Pr(>|Test stat|)

```
TempC      3.0485      0.011076 *
RtoC      -2.2847      0.043177 *
I(TempC * RtoC)  0.9899      0.343482
Tukey test   3.0857      0.002031 **
```

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

```
> ols_regress(Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC*RtoC),
+ C2H2C)
```

## Model Summary

|                |       |           |        |
|----------------|-------|-----------|--------|
| R              | 0.997 | RMSE      | 0.843  |
| R-Squared      | 0.995 | MSE       | 0.711  |
| Adj. R-Squared | 0.992 | Coef. Var | 2.954  |
| Pred R-Squared | 0.986 | AIC       | 53.943 |
| MAE            | 0.667 | SBC       | 59.351 |

RMSE: Root Mean Square Error  
MSE: Mean Square Error  
MAE: Mean Absolute Error  
AIC: Akaike Information Criteria  
SBC: Schwarz Bayesian Criteria

## ANOVA

|            | Sum of Squares | DF | Mean Square | F       | Sig.   |
|------------|----------------|----|-------------|---------|--------|
| Regression | 2112.337       | 5  | 422.467     | 371.493 | 0.0000 |

```
Residual      11.372      10      1.137
Total        2123.709      15
```

| Parameter Estimates |        |            |           |        |       |        |        |
|---------------------|--------|------------|-----------|--------|-------|--------|--------|
| model               | Beta   | Std. Error | Std. Beta | t      | Sig   | lower  | upper  |
| (Intercept)         | 36.434 | 0.553      |           | 65.899 | 0.000 | 35.202 | 37.666 |
| TempC               | 0.130  | 0.004      | 0.884     | 35.828 | 0.000 | 0.122  | 0.139  |
| RtoC                | 0.480  | 0.059      | 0.228     | 8.192  | 0.000 | 0.349  | 0.611  |
| I (TempC^2)         | 0.000  | 0.000      | 0.077     | 3.044  | 0.012 | 0.000  | 0.000  |
| I (RtoC^2)          | -0.024 | 0.010      | -0.070    | -2.324 | 0.043 | -0.046 | -0.001 |
| I (TempC * RtoC)    | -0.007 | 0.001      | -0.249    | -9.177 | 0.000 | -0.009 | -0.006 |

```
> lm.C2H2C2 <- lm(Yld ~
+ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC*RtoC), C2H2C)
> opar <- par(mfrow = c(2, 2), oma = c(0, 0, 1.1, 0))
> plot(lm.C2H2C2); par(opar)
> summary(lm.C2H2C2)

Call:
lm(formula = Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC *
RtoC), data = C2H2C)
```

```
Residuals:
    Min       1Q   Median       3Q      Max
-1.57672 -0.44853  0.02746  0.58530  1.52180
```

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.643e+01  5.529e-01  65.899 1.58e-14 ***
TempC        1.305e-01  3.642e-03  35.828 6.81e-12 ***
RtoC         4.801e-01  5.860e-02   8.192 9.56e-06 ***
I (TempC^2)   1.782e-04  5.854e-05   3.044  0.0124 *
I (RtoC^2)   -2.367e-02  1.019e-02  -2.324  0.0425 *
I (TempC * RtoC) -7.335e-03  7.993e-04  -9.177 3.47e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.066 on 10 degrees of freedom
Multiple R-squared:  0.9946,    Adjusted R-squared:  0.992
F-statistic: 371.5 on 5 and 10 DF,  p-value: 5.13e-11
```

```
> ols_coll_diag(lm.C2H2C2)
Tolerance and Variance Inflation Factor
```

|   | Variables        | Tolerance | VIF      |
|---|------------------|-----------|----------|
| 1 | TempC            | 0.8794536 | 1.137070 |
| 2 | RtoC             | 0.6886196 | 1.452181 |
| 3 | I (TempC^2)      | 0.8329067 | 1.200615 |
| 4 | I (RtoC^2)       | 0.5817242 | 1.719028 |
| 5 | I (TempC * RtoC) | 0.7279861 | 1.373653 |

Eigenvalue and Condition Index

|   | Eigenvalue  | Condition Index | intercept    | TempC       | RtoC        | I (TempC^2) |
|---|-------------|-----------------|--------------|-------------|-------------|-------------|
| 1 | 2.5152323   | 1.000000        | 0.0281686457 | 0.001598927 | 0.002564395 | 0.036071368 |
| 2 | 1.3682357   | 1.355840        | 0.0004835149 | 0.220326564 | 0.253106837 | 0.016043155 |
| 3 | 0.9286352   | 1.645760        | 0.0401626769 | 0.272243903 | 0.068540246 | 0.025323048 |
| 4 | 0.7097832   | 1.882462        | 0.0047974104 | 0.435796041 | 0.188322778 | 0.002500209 |
| 5 | 0.3518287   | 2.673766        | 0.0147320324 | 0.013270243 | 0.341321364 | 0.355512099 |
| 6 | 0.1262850   | 4.462860        | 0.9116557197 | 0.056764321 | 0.146144379 | 0.564550122 |
|   |             |                 |              |             |             | I (RtoC^2)  |
| 1 | 0.036180600 | 0.036455692     |              |             |             |             |
| 2 | 0.020061490 | 0.001214635     |              |             |             |             |
| 3 | 0.006450962 | 0.274518612     |              |             |             |             |
| 4 | 0.004308530 | 0.411182307     |              |             |             |             |
| 5 | 0.371583350 | 0.052123535     |              |             |             |             |
| 6 | 0.561415067 | 0.224505219     |              |             |             |             |

```
> anova(lm.C2H2C2)
Analysis of Variance Table
```

Response: Yld

|                 | Df | Sum Sq  | Mean Sq | F value   | Pr(>F)    |     |
|-----------------|----|---------|---------|-----------|-----------|-----|
| TempC           | 1  | 1896.68 | 1896.68 | 1667.8278 | 1.855e-12 | *** |
| RtoC            | 1  | 56.30   | 56.30   | 49.5084   | 3.557e-05 | *** |
| I(TempC^2)      | 1  | 1.11    | 1.11    | 0.9767    | 0.3463    |     |
| I(RtoC^2)       | 1  | 62.48   | 62.48   | 54.9437   | 2.282e-05 | *** |
| I(TempC * RtoC) | 1  | 95.77   | 95.77   | 84.2104   | 3.473e-06 | *** |
| Residuals       | 10 | 11.37   | 1.14    |           |           |     |

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.C2H2C2, "I(TempC^2)")
```

Linear hypothesis test:  
I(TempC^2) = 0

Model 1: restricted model  
Model 2: Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC \* RtoC)

|   | Res.Df | RSS    | Df | Sum of Sq | F      | Pr(>F)    |
|---|--------|--------|----|-----------|--------|-----------|
| 1 | 11     | 21.910 |    |           |        |           |
| 2 | 10     | 11.372 | 1  | 10.538    | 9.2669 | 0.01237 * |

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.C2H2C2, "I(RtoC^2)")
```

Linear hypothesis test:  
I(RtoC^2) = 0

Model 1: restricted model  
Model 2: Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC \* RtoC)

|   | Res.Df | RSS    | Df | Sum of Sq | F      | Pr(>F)    |
|---|--------|--------|----|-----------|--------|-----------|
| 1 | 11     | 17.512 |    |           |        |           |
| 2 | 10     | 11.372 | 1  | 6.1402    | 5.3993 | 0.04251 * |

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.C2H2C2, "I(TempC * RtoC)")
```

Linear hypothesis test:  
I(TempC \* RtoC)

Model 1: restricted model  
Model 2: Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC \* RtoC)

|   | Res.Df | RSS     | Df | Sum of Sq | F     | Pr(>F)        |
|---|--------|---------|----|-----------|-------|---------------|
| 1 | 11     | 107.137 |    |           |       |               |
| 2 | 10     | 11.372  | 1  | 95.765    | 84.21 | 3.473e-06 *** |

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.C2H2C2, c("I(TempC^2)", "I(RtoC^2)"))
```

Linear hypothesis test:  
I(TempC^2) = 0  
I(RtoC^2) = 0

Model 1: restricted model  
Model 2: Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC \* RtoC)

|   | Res.Df | RSS    | Df | Sum of Sq | F      | Pr(>F)      |
|---|--------|--------|----|-----------|--------|-------------|
| 1 | 12     | 32.308 |    |           |        |             |
| 2 | 10     | 11.372 | 2  | 20.936    | 9.2049 | 0.005403 ** |

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.C2H2C2, c("I(TempC^2)", "I(TempC * RtoC)"))
```

Linear hypothesis test:  
I(TempC^2) = 0  
I(TempC \* RtoC)

Model 1: restricted model  
Model 2: Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC \* RtoC)

|   | Res.Df | RSS     | Df | Sum of Sq | F | Pr(>F) |
|---|--------|---------|----|-----------|---|--------|
| 1 | 12     | 107.282 |    |           |   |        |

```

2      10  11.372  2      95.909 42.169 1.338e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.C2H2C2, c("I(RtoC^2)", "I(TempC * RtoC)"))

Linear hypothesis test:
I(RtoC^2) = 0
I(TempC * RtoC)

Model 1: restricted model
Model 2: Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC * RtoC)

   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
1      12 169.620
2      10  11.372  2      158.25 69.577 1.355e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.C2H2C2, c("I(TempC^2)", "I(RtoC^2)", "I(TempC * RtoC)"))

Linear hypothesis test:
I(TempC^2) = 0
I(RtoC^2) = 0
I(TempC * RtoC)

Model 1: restricted model
Model 2: Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC * RtoC)

   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
1      13 170.731
2      10  11.372  3      159.36 46.71 3.449e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> lht(lm.C2H2C2,
+ c("TempC", "RtoC", "I(TempC^2)", "I(RtoC^2)", "I(TempC * RtoC)"))

Linear hypothesis test:
TempC = 0
RtoC = 0
I(TempC^2) = 0
I(RtoC^2) = 0
I(TempC * RtoC)

Model 1: restricted model
Model 2: Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC * RtoC)

   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
1      15 2123.71
2      10   11.37  5      2112.3 371.49 5.13e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
> fitted(lm.C2H2C2)
      1      2      3      4      5      6      7      8
49.43583 49.49110 49.39908 49.01771 47.98664 44.86964 29.53812 31.42552
      9     10     11     12     13     14     15     16
33.95590 35.40819 36.94424 38.22803 13.47820 16.97922 22.07672 29.46586
> predict(lm.C2H2C2, C2H2C, T)
$fit
      1      2      3      4      5      6      7      8
49.43583 49.49110 49.39908 49.01771 47.98664 44.86964 29.53812 31.42552
      9     10     11     12     13     14     15     16
33.95590 35.40819 36.94424 38.22803 13.47820 16.97922 22.07672 29.46586

$se.fit
      1      2      3      4      5      6      7      8
0.7127879 0.5647116 0.4886780 0.5177247 0.5911888 0.8804792 0.6856833 0.5180883
      9     10     11     12     13     14     15     16
0.5316267 0.5673331 0.5461326 0.8580478 0.7489967 0.5963660 0.5820939 0.8553741

$df
[1] 10

$residual.scale
[1] 1.066402

```

```

> resid(lm.C2H2C2)
      1      2      3      4      5      6
-0.43582874  0.70889749  1.10092023 -0.51771385 -0.48663827 -0.36963687
      7      8      9     10     11     12
-1.53811898  0.07448063  0.54410333 -0.40819238  1.05575694  0.27197045
     13     14     15     16
  1.52180232  0.02077970 -1.57672387  0.03414185
> confint(lm.C2H2C2)
              2.5 %          97.5 %
(Intercept)    3.520194e+01 37.6657029171
TempC          1.223630e-01  0.1385917617
RtoC           3.494877e-01  0.6106350156
I(TempC^2)      4.776824e-05  0.0003086328
I(RtoC^2)      -4.637588e-02 -0.0009730083
I(TempC * RtoC) -9.115543e-03 -0.0055537503
> # EXAMPLE 6-9 (continued)
> # § 6-4.2 Categorical Regressors
> # EXAMPLE 6-10 Gas Mileage
> # Use Indicator Variables to Model Interaction
> # EXAMPLE 6-11 Shampoo Data
> # Refer to § 2-6.
> # § 6-4.3 Variable Selection Techniques
> # EXAMPLE 6-12 All Possible Regressions
> # Table 6-10 Minitab Best Subsets Regression for Shampoo Data
> # EXAMPLE 6-13 Stepwise Regression
> # Table 6-11 Stepwise Regression Backward Elimination for Shampoo Data:
> # Quality versus Foam, Scent, Color, Residue, Region
> # Table 6-12 Stepwise Regression Forward Selection for Shampoo Data:
> # Quality versus Foam, Scent, Color, Residue, Region
> # Table 6-13 Stepwise Regression Combined Forward and Backward Elimination:
> # Quality versus Foam, Scent, Color, Residue, Region
> # EXERCISES FOR § 6-4
>

```